

# constitution\_updates

## LLMsVerifier Integration — Constitution & Documentation Updates

**Status:** Draft for insertion into HelixCode, HelixAgent, and ALL submodules  
**Mandate:** User explicitly demands: “This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules’s Constitution, CLAUDE.MD and AGENTS.MD as well.”  
**Version:** 1.0.0-Draft  
**Date:** 2026-07-01

### Table of Contents

- 1. [Executive Summary](#)
- 2. [CONSTITUTION.md Amendments](#)
- 3. [CLAUDE.md Updates](#)
- 4. [AGENTS.md Updates](#)
- 5. [Configuration Documentation](#)
- 6. [User Guide](#)
- 7. [Integration Guide for Developers](#)
- 8. [Submodule Constitution Template](#)

### 1. Executive Summary

This document specifies the **exact content** that must be inserted into CONSTITUTION . md, CLAUDE . md, AGENTS . md, and all submodule equivalents to codify LLMsVerifier integration as **project law**. The following gaps in HelixCode have been identified from analysis:

| Gap                          | Current State                                         | Required State                                       |
|------------------------------|-------------------------------------------------------|------------------------------------------------------|
| BLUFF-002                    | cmd/cli/main.go:101-128 hardcodes 3 models            | Must fetch from LLMsVerifier                         |
| BLUFF-001                    | Simulated LLM responses in .old file                  | All generation must route through verified providers |
| No Dynamic Model Source      | model_discovery.go hardcodes external models          | Must use LLMsVerifier as single source of truth      |
| No Verifier Config           | No LLMsVerifier section in config.go                  | Full verifier YAML schema required                   |
| No Anti-Bluff Test Guarantee | CONST-035 exists but lacks verifier-specific language | Must mandate tests that prove features actually work |

| Gap                 | Current State                             | Required State                 |
|---------------------|-------------------------------------------|--------------------------------|
| Submodule Alignment | Submodules have independent constitutions | Must propagate all rules below |

**Legal Force:** Every rule in Section 2 carries the same binding authority as CONST-001 through CONST-035. Violation is a constitutional breach.

---

## 2. CONSTITUTION.md Amendments

### 2.1 Insert After CONST-035 — New Rules CONST-036 through CONST-040

**Insertion Point:** Immediately after the closing block of CONST-035 in CONSTITUTION.md (HelixCode) and equivalent location in all submodule CONSTITUTION.md files.

---

### CONST-036: LLMsVerifier Single Source of Truth Mandate

**\*\*Rule\*\*:** LLMsVerifier SHALL BE the sole authoritative source for:

1. All model metadata (names, IDs, context windows, capabilities)
2. All provider metadata (endpoints, auth types, supported models)
3. All verification status (verified, partial, failed, pending)
4. All scoring data (overall scores, capability scores, tier rankings)
5. All rate-limit and cooldown state

**\*\*Prohibition\*\*:** NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

**\*\*Anti-Bluff Verification\*\*:**

- The challenge script `challenges/scripts/llmsverifier\_hardcode\_check.sh` MUST scan all Go source files for hardcoded model arrays.
- Any `[string{"gpt-4", "claude-3"}]` or equivalent literal in production code is a constitutional violation.
- The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

**\*\*Enforcement\*\***: ``make test-complete`` MUST include a test that asserts ``ModelManager.GetAvailableModels()`` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-017.

---

### ### CONST-037: Model Provider Anti-Bluff Guarantee

**\*\*Rule\*\***: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last ``verification_timeout`` period (default: 24h). Models older than this MUST display a "stale" indicator and be deprioritized.

**\*\*Prohibition Against Test Bluffing\*\***:

- A unit test that mocks the verifier client and asserts ``GetAvailableModels()`` returns 3 models DOES NOT satisfy this rule.
- An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response DOES satisfy this rule.
- The Makefile target ``make test-verifier-integration`` MUST exist and MUST run without mocks.

**\*\*The "Tests Pass But Features Don't Work" Guarantee\*\*** (User-Demand Anti-Bluff):

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If ``TestModelList`` passes but ``helixcode --list-models`` shows hardcoded
- If ``TestProviderHealth`` passes but the health endpoint returns ``200 OK``
- If ``TestLLMGeneration`` passes but ``--prompt "hello"`` returns a simulated
- Bluff tests MUST be rewritten or deleted. There is no "grandfather" exception.

**\*\*Evidence Standard\*\***: Every test that claims to verify model/provider fun

1. Call a real API endpoint or a real verifier database
2. Assert on response content that could only come from that real source
3. Include a ``t.Parallel()`` integration test that runs the CLI binary with

---

### ### CONST-038: Real-Time Model Status Accuracy

**\*\*Rule\*\***: Model status (available, rate-limited, cooldown, offline, deprec

**\*\*Polling vs. Push\*\***:

- If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier
- The TUI MUST display a "last updated" timestamp with every model listing
- Models in "cooldown" or "rate-limited" state MUST show the estimated rec

**\*\*Accuracy Verification\*\*:**

- Challenge script `challenges/scripts/model\_status\_accuracy\_challenge.sh`
  1. Artificially rate-limit a provider by exhausting its quota
  2. Wait for the status to propagate to the verifier
  3. Check that `helixcode --list-models` shows the rate-limited status wi
  4. Check that `SelectOptimalModel()` no longer selects the rate-limited

**\*\*Prohibition\*\*:** Status indicators that are "always green" or that lag >60

---

### ### CONST-039: All Providers and Models Integration Mandate

**\*\*Rule\*\*:** HelixCode MUST integrate with ALL providers and models that LLMs

1. The provider being explicitly disabled in configuration (`enabled: fals
2. The API key being absent and the provider requiring one
3. The provider being marked `deprecated` in the verifier database

**\*\*Minimum Provider Set\*\*** (SHALL NOT be reduced without constitutional amen

| Provider              | Auth Type            | Required Env Var            |
|-----------------------|----------------------|-----------------------------|
| OpenAI                | API Key              | `HELIX_OPENAI_API_KEY`      |
| Anthropic             | API Key / OAuth      | `HELIX_ANTHROPIC_API_KEY`   |
| Gemini                | API Key              | `HELIX_GEMINI_API_KEY`      |
| DeepSeek              | API Key              | `HELIX_DEEPSEEK_API_KEY`    |
| Groq                  | API Key              | `HELIX_GROQ_API_KEY`        |
| Together AI           | API Key              | `HELIX_TOGETHER_API_KEY`    |
| Mistral               | API Key              | `HELIX_MISTRAL_API_KEY`     |
| xAI                   | API Key              | `HELIX_XAI_API_KEY`         |
| Cerebras              | API Key              | `HELIX_CEREBRAS_API_KEY`    |
| Cloudflare Workers AI | API Key + Account ID | `HELIX_CLOUDFLARE_API_KEY`  |
| SiliconFlow           | API Key              | `HELIX_SILICONFLOW_API_KEY` |
| Replicate             | Token                | `HELIX_REPLICATE_API_TOKEN` |
| OpenRouter            | API Key              | `HELIX_OPENROUTER_API_KEY`  |
| Ollama                | Local                | None (auto-detect)          |
| Llama.cpp             | Local                | None (auto-detect)          |

**\*\*Integration Requirement\*\*:** For every provider in the minimum set:

- There MUST be a provider adapter file in `internal/llm/` or `internal/ve
- There MUST be a `\*\_test.go` file with real API tests (skipped only if `H
- There MUST be a challenge script in `challenges/scripts/`
- The model listing MUST include models from this provider when the provid

---

### ### CONST-040: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integr

**\*\*Rule\*\*:** LLMsVerifier integration SHALL extend beyond basic model listing

1. **\*\*MCP (Model Context Protocol)\*\*:** The verifier MUST report which models

2. **LSP (Language Server Protocol)**: The verifier MUST report code-analysis
3. **ACP (Agent Capability Protocol)**: The verifier MUST report multi-agent
4. **Embedding**: The verifier MUST report `supports_embeddings` for each
5. **RAG (Retrieval-Augmented Generation)**: The verifier MUST report context
6. **Skills / Plugins**: The verifier MUST track plugin compatibility. Model

**Capability Checklist** (MUST be verified by challenge `challenges/script`)

- [ ] MCP tool calling verified for at least 3 providers
- [ ] LSP code-analysis verified for at least 3 providers
- [ ] ACP parallel tool use verified for at least 2 providers
- [ ] Embedding generation verified for at least 2 providers
- [ ] RAG context-window adaptation verified
- [ ] Skills/plugin execution verified for at least 2 providers

**Prohibition**: Capability flags MUST NOT be hardcoded. The `Provider.Get`

## 2.2 Amendment to CONST-035 (End-User Usability Mandate)

**Insertion Point:** Within the body of CONST-035, add the following paragraph after the existing anti-bluff language and before the closing statement:

**LLMsVerifier Usability Extension**: The "End-User Usability Mandate" explicitly requires that every model listing, selection, and generation feature MUST be usable with real, verified models. The following specific behaviors are considered USABILITY FAILURES under this rule:

1. The `--list-models` flag displays fewer models than the verifier has discovered for enabled providers.
2. The `--list-models` flag displays models that the verifier has marked `failed` or `unavailable` without indicating their status.
3. The `--prompt` flag uses a hardcoded or simulated response when a real provider is configured and available.
4. The TUI model selection screen shows "no models available" while the verifier database contains verified models.
5. The `SelectOptimalModel()` function selects a model that the verifier has scored below the configured `min_score` threshold.
6. API keys are present in environment variables but the corresponding provider is not listed because the discovery code is not implemented.

Any PASS that exhibits any of these behaviors is a BLUFF PASS and violates both CONST-035 and CONST-037.

## 2.3 Amendment to CONST-017 (Zero-Bluff Testing)

**Insertion Point:** Within CONST-017, append the following clause:

**\*\*LLMsVerifier Zero-Bluff Clause\*\*:** A test that verifies model or provider behavior is a BLUFF unless:

1. It calls the LLMsVerifier client with ``testMode: false`` (or the production verifier endpoint), OR
2. It queries the verifier SQLite database directly and asserts on real rows, OR
3. It invokes the CLI binary as a subprocess and asserts on stdout/stderr output that must originate from the verifier.

Mocking the verifier client with hardcoded

``VerificationResult{OverallScore: 8.5}`` is forbidden in all test tiers above unit tests. The ``verification.go`` stub in the verifier submodule (which returns hardcoded 8.5 scores) MUST NOT be the data source for any HelixCode test.

## 2.4 New Appendix to CONSTITUTION.md — “LLMsVerifier Compliance Manifest”

**Insertion Point:** At the end of CONSTITUTION.md, before any existing “End of Document” marker.

---

### ## Appendix D: LLMsVerifier Compliance Manifest

This appendix codifies the files, tests, and scripts that MUST exist for constitutional compliance.

#### ### D.1 Required Files

| File                                          | Purpose                          | Constitutional Rule |
|-----------------------------------------------|----------------------------------|---------------------|
| -----                                         | -----                            | -----               |
| <code>`internal/verifier/service.go`</code>   | VerificationService wrapper      | CONST-036           |
| <code>`internal/verifier/config.go`</code>    | Verifier Config structs          | CONST-039           |
| <code>`internal/verifier/discovery.go`</code> | ModelDiscoveryService            | CONST-036           |
| <code>`internal/verifier/startup.go`</code>   | StartupVerifier 5-phase pipeline | CONST-039           |
| <code>`internal/verifier/scoring.go`</code>   | ScoringService adapter           | CONST-036           |
| <code>`internal/verifier/health.go`</code>    | Health monitoring                | CONST-038           |

```
| `internal/verifier/events.go` | Event bus integration |  
    CONST-038 |  
| `configs/verifier.yaml` | Full verifier configuration |  
    CONST-039 |  
| `pkg/sdk/go/verifier/client.go` | Go SDK client for verifier |  
    CONST-036 |  
| `docs/guides/llms-verifier.md` | User-facing integration guide  
    | CONST-035 |  
| `docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md` | Developer  
    integration plan | CONST-037 |  
| `challenges/scripts/llmsverifier_hardcode_check.sh` | Hardcoded  
    model scan | CONST-036 |  
| `challenges/scripts/llmsverifier_capabilities_challenge.sh` |  
    Capability verification | CONST-040 |  
| `challenges/scripts/llmsverifier_status_accuracy_challenge.sh`  
    | Status accuracy test | CONST-038 |  
| `LLMsVerifier/` (submodule) | Points to `vasic-digital/  
    LLMsVerifier` | CONST-036 |
```

### ### D.2 Required Makefile Targets

```
```makefile  
test-verifier-unit:          ## Unit tests for verifier package  
    (mocks OK)  
test-verifier-integration:   ## Integration tests with real  
    verifier DB (NO MOCKS)  
test-verifier-e2e:          ## End-to-end verifier challenges  
test-verifier-status:       ## Status accuracy challenge  
test-verifier-capabilities:  ## Capability verification challenge  
test-verifier-no-  
    mocks:                  ## Fails if any non-unit test uses a mock  
test-verifier-hardcode:     ## Fails if hardcoded models found  
    in production code
```

### D.3 Required Environment Variables (in .env.example)

```
# LLMsVerifier Core  
HELIX_VERIFIER_ENABLED=true  
HELIX_VERIFIER_ENDPOINT=http://localhost:8081  
HELIX_VERIFIER_API_KEY=  
HELIX_VERIFIER_TIMEOUT=30s  
HELIX_VERIFIER_DATABASE_PATH=./data/llm-verifier.db  
HELIX_VERIFIER_ENCRYPTION_KEY=  
HELIX_VERIFIER_JWT_SECRET=  
  
# Provider API Keys  
HELIX_OPENAI_API_KEY=  
HELIX_ANTHROPIC_API_KEY=
```

```
HELIX_GEMINI_API_KEY=
HELIX_DEEPSEEK_API_KEY=
HELIX_GROQ_API_KEY=
HELIX_TOGETHER_API_KEY=
HELIX_MISTRAL_API_KEY=
HELIX_XAI_API_KEY=
HELIX_CEREBRAS_API_KEY=
HELIX_CLOUDFLARE_API_KEY=
HELIX_CLOUDFLARE_ACCOUNT_ID=
HELIX_SILICONFLOW_API_KEY=
HELIX_REPLICATE_API_TOKEN=
HELIX_OPENROUTER_API_KEY=
HELIX_QWEN_API_KEY=
HELIX_COHERE_API_KEY=
```

#### # Local Providers

```
HELIX_OLLAMA_HOST=http://localhost:11434
HELIX_LLAMA_CPP_HOST=http://localhost:8080
```

---

### ## 3. CLAUDE.md Updates

#### ### 3.1 Insert New Section – "LLMsVerifier Integration Architecture"

**\*\*Insertion Point\*\***: After the existing "Architecture Overview" or "Core S

``markdown

---

### ## LLMsVerifier Integration Architecture

#### ### Overview

LLMsVerifier is the **\*\*single source of truth\*\*** for all model and provider

**\*\*Philosophy\*\***: No code in HelixCode should ever hardcode a model name, pr

#### ### System Diagram

```
+-----+ REST/WebSocket +-----+ | HelixCode | <-----> |
LLMsVerifier | | (Main Process) | /api/v1/verifier | (Submodule) | +-----+ +
+-----+ + | | imports | manages v v +-----+ + +-----+ + | internal/verifier|
| SQLite DB | | - service.go | | - models | | - discovery.go | | - providers | | - startup.go | | -
verification | | - scoring.go | | results | | - health.go | | - limits | +-----+ + +-----+
| ^ | calls real APIs | reads v | +-----+ + +-----+ + | Provider APIs | | Provider
Adapters | | (OpenAI, etc.) | | (12+ adapters) | +-----+ + +-----+ +
```

#### ### Key Components



#### VerificationService (`internal/verifier/service.go`)

This is the primary interface between HelixCode and LLMsVerifier. It wraps

```
```go
func (vs *VerificationService) GetVerifiedModels(provider string) ([]*Unif
func (vs *VerificationService) GetModelScore(modelID string) (float64, boo
func (vs *VerificationService) IsModelAvailable(modelID string) (bool, tim
func (vs *VerificationService) ValidateResponseQuality(content string, lat
func (vs *VerificationService) IsCannedErrorResponse(content string) (bool
```

**Anti-Bluff:** ValidateResponseQuality checks for canned responses and suspiciously fast replies (<100ms). Any model that passes this check is marked as verified. Models that fail are marked as failed and excluded from selection.

### ModelDiscoveryService (internal/verifier/discovery.go)

Runs the 5-phase startup pipeline: 1. **Discover:** Scan environment for API keys, OAuth tokens, local endpoints 2. **Verify:** Call each provider’s API to confirm model existence and responsiveness 3. **Detect Subscriptions:** Determine 3-tier subscription level (Premium/High-quality/Fast) 4. **Score:** Run the 7-component scoring engine (code capability, responsiveness, reliability, feature richness, value proposition, cost effectiveness, recency) 5. **Rank:** Sort providers and models by score, select debate team

```
type DiscoveryConfig struct {
    Enabled                bool                `yaml:"enabled"`
    DiscoveryInterval      time.Duration      `yaml:"discovery_interval"`
    MaxModelsForEnsemble  int                `yaml:"max_models_for_ensemble"`
    MinScore               float64            `yaml:"min_score"`
    RequireVerification    bool               `yaml:"require_verification"`
    RequireCodeVisibility  bool               `yaml:"require_code_visibility"`
    RequireDiversity       bool               `yaml:"require_diversity"`
    ProviderPriority       []string           `yaml:"provider_priority"`
}
```

### ScoringService (internal/verifier/scoring.go)

The scoring engine evaluates models across 7 components with configurable weights:

| Component       | Default Weight | Description                  |
|-----------------|----------------|------------------------------|
| Code Capability | 40%            | Coding task success rate     |
| Responsiveness  | 20%            | Average latency, P95 latency |
| Reliability     | 20%            | Uptime, error rate           |

| Component         | Default Weight | Description                       |
|-------------------|----------------|-----------------------------------|
| Feature Richness  | 15%            | Tool use, streaming, vision, etc. |
| Value Proposition | 5%             | Cost per token, open-source bonus |

**Customizing Weights:** Edit `configs/verifier.yaml` under `verifier.scoring.weights`. The weights must sum to 1.0 (validated at startup).

### StartupVerifier (internal/verifier/startup.go)

The master orchestrator. Implements `VerifyAllProviders()` with circuit breaker, faulty key deprioritization, and OAuth fallback.

#### Critical Method:

```
func (sv *StartupVerifier) VerifyAllProviders(ctx
    context.Context) (*StartupResult, error)
```

This method is called during HelixCode server startup (in `cmd/server/main.go` after config load) and during CLI `--list-models` when cache is stale.

## Integration Patterns

### Pattern 1: Direct Verifier Client (For New Features)

When adding a feature that needs model data, use the Go SDK client:

```
import "github.com/HelixDevelopment/helix_code/pkg/sdk/go/
    verifier"
```

```
client := verifier.New(verifier.ClientConfig{
    BaseURL: "http://localhost:8081",
    APIKey:  os.Getenv("HELIX_VERIFIER_API_KEY"),
    Timeout: 30 * time.Second,
})
```

```
models, err := client.GetModels(ctx)
scores, err := client.GetProviderScores(ctx)
```

### Pattern 2: Score Adapter (For Provider Selection)

When the existing `ModelManager.SelectOptimalModel()` needs verifier scores, use the adapter:

```
import "github.com/HelixDevelopment/helix_code/internal/services"
```

```
adapter := services.NewLLMsVerifierScoreAdapter(
    verifierScoringService,
    verificationService,
    logger,
```

```
)

score, ok := adapter.GetProviderScore("openai")
if ok && score > 7.0 {
    // Prefer this provider
}
```

### Pattern 3: Event-Driven Updates (For Real-Time UI)

Subscribe to verifier events via WebSocket:

```
wsURL := "ws://localhost:8081/ws/verifier/events"
conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
// Read VerificationEvent structs
```

Event types: `verification.started`, `verification.provider.discovered`, `verification.provider.verified`, `verification.provider.failed`, `verification.provider.scored`, `verification.debate_team.selected`, `verification.completed`.

### How to Add a New Provider Through the Verifier

1. **Add Provider Adapter** in `LLMsVerifier` submodule:
  - Create `llm-verifier/providers/newprovider.go`
  - Embed `BaseAdapter`
  - Implement `SendRequest()`, `ParseResponse()`
  - Add fallback models to `fallback_models.go`
2. **Add to HelixCode Provider Registry:**
  - Add entry to `internal/verifier/provider_types.go` in `SupportedProviders` map
  - Set `AuthType`, `Tier`, `Priority`, `EnvVars`, `BaseURL`
  - Add env var mapping to `.env.example`
3. **Add API Key Support:**
  - Add `HELIX_NEWPROVIDER_API_KEY` to `.env.example`
  - Add to `internal/verifier/startup.go` discovery logic
  - Document in `docs/guides/llms-verifier.md`
4. **Add Tests:**
  - `internal/verifier/adapters/newprovider_test.go` (unit)
  - `tests/integration/newprovider_verification_test.go` (integration)
  - `challenges/scripts/newprovider_challenge.sh` (E2E)
5. **Add to Config Schema:**
  - Add `newprovider` section to `configs/verifier.yaml`
  - Add to `internal/verifier/config.go` `Config` struct
6. **Update ModelManager:**
  - Ensure `ModelManager` can read models from verifier for this provider
  - No hardcoded model list needed — verifier provides it

## API Key Provisioning

### Environment Variable Discovery

At startup, `StartupVerifier` scans environment variables using `api_keys.NewEnvVarScanner()`:

```
scanner := api_keys.NewEnvVarScanner()
keys, err := scanner.ScanEnvForUnsupportedKeys()
// Returns map[providerType]apiKeyValue
```

**Priority Order** (from `internal/verifier/startup.go`): 1. Non-faulty API keys first (faulty keys are deprioritized) 2. OAuth tokens second (if `OAuthPrimaryNonOAuthFallback: true`) 3. Free providers third 4. Local providers last (checked via localhost probes)

### OAuth Support

For providers requiring OAuth (e.g., Claude, Qwen): - `oauth_credentials.OAuthCredentialReader` reads tokens from secure storage - Tokens are refreshed automatically before expiry - `TrustOAuthOnFailure: true` falls back to API key on OAuth failure

### API Key Redaction

API keys are NEVER serialized to JSON or logged:

```
type UnifiedProvider struct {
    APIKey string `json:"- "` // Never serialized
}
```

## Debugging Verifier Integration Issues

### Diagnostic Commands

```
# Check verifier health
```

```
helixcode verifier health
```

```
# List discovered providers with scores
```

```
helixcode verifier providers list --format json
```

```
# Check why a provider is not available
```

```
helixcode verifier providers get openai --verbose
```

```
# Run verification for a single model
```

```
helixcode verifier models verify gpt-4o
```

```
# Export full verifier config
```

```
helixcode verifier config export yaml
```

### # Check event log

```
helixcode verifier events list --limit 20
```

### # Check rate limits

```
helixcode verifier limits list
```

## Common Issues

| Symptom                                  | Cause                                   | Fix                                                                  |
|------------------------------------------|-----------------------------------------|----------------------------------------------------------------------|
| --list-models shows empty list           | Verifier not started                    | Start verifier: <code>helixcode verifier start</code>                |
| --list-models shows 3 hardcoded models   | BLUFF-002 not fixed                     | Replace <code>handleListModel</code> s with verifier fetch           |
| Provider missing despite API key present | Key marked faulty                       | Run <code>api_keys.ClearFaultyKey()</code> or fix key                |
| Score is always 8.5                      | Using stub <code>verification.go</code> | Ensure real <code>coding_capability_verification.go</code> is active |
| Status not updating                      | WebSocket down / polling off            | Check <code>events.websocket.enabled</code> in config                |
| OAuth provider failing                   | Token expired                           | Check OAuth credential reader logs                                   |

## Log Levels

Set `HELIX_VERIFIER_LOG_LEVEL=debug` to see: - Every provider discovery attempt - Every verification request/response - Every scoring calculation - Every circuit breaker state change

## Real-Time Updates Architecture

### WebSocket Event Stream

```
events:
  websocket:
    enabled: true
    path: "/ws/verifier/events"
```

The verifier publishes events to WebSocket clients. HelixCode subscribes and updates: - Model status in TUI - Provider health in dashboard - Score badges in CLI output

### Polling Fallback

When WebSocket is unavailable:

```
pollInterval := config.Events.PollInterval // default 30s
```

The `ModelDiscoveryService` polls the verifier REST API (GET `/api/v1/verifier/models`) on this interval.

## TUI Update Strategy

The TUI uses `bubbletea` with a `TickMsg` every `pollInterval`:

```
type TickMsg time.Time

func (m Model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
    switch msg.(type) {
    case TickMsg:
        return m, tea.Batch(m.refreshModels(), m.tick())
    }
}
```

## UX Guidelines for Model Display

### Score Suffix Format

Append `SC:X.X` to model display names:

```
✓ GPT-4o SC:9.2 [Tier 1] [Code ✓] [Vision ✓]
○ GPT-4 SC:8.8 [Tier 1] [Code ✓] [Stale 2h]
× GPT-3.5 SC:7.1 [Tier 3] [Rate Limited 15m]
```

### Status Indicators

| Indicator                                                                           | Meaning                                    |
|-------------------------------------------------------------------------------------|--------------------------------------------|
| ✓                                                                                   | Verified, score > min_score                |
| ○                                                                                   | Verified but stale (>24h since last check) |
| ×                                                                                   | Failed verification or rate limited        |
|  | Verification in progress                   |
|  | Code visibility confirmed                  |
|  | Premium tier (1-2)                         |
|  | Fast tier (3)                              |
|  | Free tier (4-5)                            |

### Sort Order

Default sort for model listings: 1. Verified first, failed last 2. By tier (1 highest, 5 lowest) 3. By score (highest first) 4. By latency (lowest first) 5. Alphabetical

## Deprecation Handling

Models with Deprecated: true in verifier data: - Show strikethrough or dimmed text -  
Display deprecation date if known - Do NOT select automatically (manual override required)

---

### ## 4. AGENTS.md Updates

#### ### 4.1 Insert New Bluff Area — BLUFF-004 through BLUFF-008

**\*\*Insertion Point\*\*:** After BLUFF-003 in `AGENTS.md`, before the "Free AI P

```markdown

---

#### ### BLUFF-004: LLMsVerifier Integration is Stubbed or Bypassed (CRITICAL)

**\*\*File Pattern\*\*:** `internal/verifier/\*.go` containing empty structs, `// T

**\*\*Evidence Standard\*\*:**

- `VerificationService` methods return hardcoded `VerificationResult{Overa
- `ModelDiscoveryService` returns an empty slice instead of calling provid
- `StartupVerifier` skips all 5 phases and returns a mock result
- The verifier submodule directory `LLMsVerifier/` exists but is empty (no

**\*\*Fix Priority\*\*:** P0 - Immediate

**\*\*Verification Command\*\*:**

```bash

make test-verifier-integration

# This MUST pass with real verifier data, not mocked scores

---

## BLUFF-005: Provider Discovery Uses Hardcoded Env Var Names (HIGH)

**File Pattern:** internal/verifier/startup.go or provider adapter files containing  
hardcoded strings like "OPENAI\_API\_KEY" without checking  
SupportedProviders[provider].EnvVars.

**Evidence Standard:** - The discovery code checks os.Getenv("OPENAI\_API\_KEY")  
directly instead of using api\_keys.GetProviderAPIKeyName("openai") - Adding a  
new provider requires modifying discovery code instead of just adding to  
SupportedProviders - Environment variable names are duplicated in multiple files

**Fix Priority:** P1 - High

**Fix Pattern:** Use the SupportedProviders map as the single source of truth for env var  
names:

```
providerInfo := SupportedProviders[providerType]
for _, envVar := range providerInfo.EnvVars {
```

```
    if key := os.Getenv(envVar); key != "" {
        return key, nil
    }
}
```

---

## BLUFF-006: Model Capabilities Are Hardcoded (HIGH)

**File Pattern:** `internal/llm/*.go` containing `SupportsToolUse: true` as a struct literal for specific models, or `Provider.GetCapabilities()` returning a static slice.

**Evidence Standard:** - `GetCapabilities()` returns `[]ModelCapability{ToolUse, CodeGeneration}` without querying verifier - The capability list for a model is written in source code rather than read from `VerificationResult.FeatureDetection` - Adding a new capability to a model requires a code change instead of a verifier re-run

**Fix Priority:** P1 - High

**Constitutional Impact:** Violates CONST-040 (MCP/LSP/ACP/Embedding/RAG/Skills/Plugins Integration Mandate).

---

## BLUFF-007: Test Claims Integration But Uses Mocked Verifier (CRITICAL)

**File Pattern:** `*_test.go` files with `testify/mock` or `testMode: true` in non-unit test files.

**Evidence Standard:** - `TestModelDiscovery` creates a mock verifier client and asserts it returns 5 models - `TestProviderSelection` stubs `GetProviderScore()` to return 9.0 - The test does not start the actual verifier server, does not create the SQLite database, and does not make real HTTP calls - The test passes in CI but the feature fails when used by a human

**Fix Priority:** P0 - Immediate

**Constitutional Impact:** Violates CONST-037 (Model Provider Anti-Bluff Guarantee) and CONST-017 (Zero-Bluff Testing).

**Required Test Structure:**

```
func TestModelDiscovery_Integration(t *testing.T) {
    if os.Getenv("HELIX_SKIP_LIVE_PROVIDER_TESTS") != "" {
        t.Skip("SKIP-OK: Live provider tests disabled")
    }

    // Start real verifier server
    srv := startVerifierServer(t)
    defer srv.Stop()

    // Configure a real provider with real API key
    cfg := loadVerifierConfig()

    // Run discovery
```



```

discovery := verifier.NewModelDiscoveryService(cfg)
models, err := discovery.DiscoverAll(ctx)

// Assert on REAL data
require.NoError(t, err)
require.Greater(t, len(models), 0, "Discovery must find at
    least one model")

// Verify each model has a real provider
for _, m := range models {
    require.NotEmpty(t, m.Provider, "Model %s must have a
        provider", m.ID)
    require.True(t, m.Verified, "Model %s must be verified",
        m.ID)
}
}

```

---

## BLUFF-008: Scoring Weights Do Not Sum to 1.0 (MEDIUM)

**File Pattern:** configs/verifier.yaml or internal/verifier/config.go where scoring weights are misconfigured.

**Evidence Standard:** - weights: {response\_speed: 0.3, model\_efficiency: 0.3, cost\_effectiveness: 0.3, capability: 0.3, recency: 0.3} sums to 1.5 - No validation at startup to check weight sum - Scores exceed 10.0 or are negative due to misweighted calculation

**Fix Priority:** P2 - Medium

**Fix:** Add validateWeights() at config load time:

```

func (c *ScoringConfig) validateWeights() error {
    sum := c.Weights.ResponseSpeed + c.Weights.ModelEfficiency +
        c.Weights.CostEffectiveness + c.Weights.Capability +
        c.Weights.Recency
    if math.Abs(sum-1.0) > 0.001 {
        return fmt.Errorf("scoring weights must sum to 1.0, got
            %.3f", sum)
    }
    return nil
}

```

|                                                                                                     |
|-----------------------------------------------------------------------------------------------------|
| ...                                                                                                 |
| ### 4.2 Insert Updated Technology Stack Reference                                                   |
| <b>Insertion Point:</b> In the “Technology Stack” section of AGENTS.md, add the verifier subsystem: |

markdown ##### LLMsVerifier  
Subsystem Stack -  
LLMsVerifier submodule (Go 1.25.3) – git submodule at `LLMsVerifier/` - SQLite 3 with WAL mode – verifier database - SQL Cipher (optional) – database encryption - Bubbletea v1.1.0 – TUI for verifier - go-playground/validator/v10 – input validation - Gorilla WebSocket v1.5.3 – real-time events - Prometheus client – metrics export

#### ### 4.3 Insert New Module Boundaries

**Insertion Point:** In the “Module Boundaries” or “Architecture” section of AGENTS.md:

``markdown ##### Verifier Module Boundaries

**Package:** internal/verifier/ — The ONLY package that directly imports the LLMsVerifier submodule.

**Allowed Dependencies:** - internal/verifier/ MAY import: digital.vasic.llmsverifier/\*, github.com/gorilla/websocket, github.com/sirupsen/logrus - internal/services/ MAY import: internal/verifier/\* (through adapter only) - internal/llm/ MAY import: internal/services/llmsverifier\_score\_adapter.go (only) - cmd/cli/ MAY import: internal/verifier/\* (for verifier subcommand) - cmd/server/ MAY import: internal/verifier/\* (for startup verification)

**Forbidden Dependencies:** - internal/verifier/ MUST NOT import: internal/llm/ (circular — verifier is below LLM layer) - internal/config/ MUST NOT import: internal/verifier/ (config is above all) - internal/llm/ MUST NOT import: digital.vasic.llmsverifier/\* directly (must go through adapter) - internal/server/ MUST NOT

|                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| import:<br>digital.vasic.llmsverifier/*<br>directly                                                                                                                                                                                                                           |
| <b>Submodule Rule:</b> The LLMsVerifier/<br>directory is a git submodule. It MUST<br>NOT be edited directly from HelixCode.<br>Changes to the verifier go through the<br>upstream vasic-digital/<br>LLMsVerifier repository and are<br>pulled via git submodule update.<br>`` |
| ### 4.4 Insert Challenge Verification<br>Checklist                                                                                                                                                                                                                            |
| <b>Insertion Point:</b> After the existing test<br>category list or at the end of AGENTS.md:                                                                                                                                                                                  |
| ``markdown                                                                                                                                                                                                                                                                    |

## LLMsVerifier Challenge Verification Checklist

Every agent working on verifier-related code MUST confirm the following before marking work complete:

### Pre-Implementation

- ☐ Read docs/integration/LLMSVERIFIER\_INTEGRATION\_PLAN.md phases 1-10
- ☐ Confirm LLMsVerifier/ submodule is initialized (git submodule status)
- ☐ Confirm configs/verifier.yaml schema matches the code being written
- ☐ Check .env.example has all required HELIX\_VERIFIER\_\* and provider API key variables

### During Implementation

- ☐ No hardcoded model lists in any modified file (run make test-verifier-hardcode)
- ☐ No hardcoded provider lists (use SupportedProviders map)
- ☐ No hardcoded capability flags (read from verifier database)
- ☐ All provider API keys use env var names from SupportedProviders[provider].EnvVars
- ☐ Scoring weights validated to sum to 1.0

☐

OAuth providers have fallback to API key if configured

☐

Circuit breaker configured for all external providers

## Post-Implementation Testing

☐

`make test-verifier-unit` passes (mocks OK for unit tests)

☐

`make test-verifier-integration` passes (NO MOCKS — real verifier DB)

☐

`make test-verifier-hardcode` passes (zero hardcoded models in production)

☐

`make test-verifier-no-mocks` passes (no non-unit test uses mock)

☐

`challenges/scripts/llmsverifier_hardcode_check.sh` passes

☐

`challenges/scripts/llmsverifier_capabilities_challenge.sh`  
passes

☐

`challenges/scripts/llmsverifier_status_accuracy_challenge.sh`  
passes

☐

`challenges/scripts/llmsverifier_startup_verification_challenge.sh` passes

☐

CLI `--list-models` output matches verifier database content

☐

`SelectOptimalModel()` never selects a model with verifier score below `min_score`

## Anti-Bluff Confirmation

☐

Run `helixcode --list-models` manually and compare to `llm-verifier models list`

☐

Run `helixcode --prompt "test" --model <verified-model>` and confirm real API call (check network traffic)

☐

Temporarily disable a provider in config and confirm it disappears from `--list-models` within 60s

☐

Verify that rate-limited models show rate-limited status in CLI output

☐

Check that models with `Deprecated: true` are NOT auto-selected

---

## ## LLMsVerifier Quick Reference for Agents

### ### Essential Commands

```
```bash
# Initialize submodule
git submodule update --init --recursive

# Start verifier server
cd LLMsVerifier/llm-verifier && go run cmd/main.go server

# Run verifier CLI
./llm-verifier models list
./llm-verifier providers list
./llm-verifier models verify MODEL_ID

# Run HelixCode with verifier
HELIX_VERIFIER_ENABLED=true ./helixcode --list-models

# Run challenges
./challenges/scripts/llmsverifier_hardcode_check.sh
./challenges/scripts/llmsverifier_startup_verification_challenge.sh
```

## Essential Files

| File                                              | When to Read                                 |
|---------------------------------------------------|----------------------------------------------|
| configs/verifier.yaml                             | Before modifying any verifier config         |
| internal/verifier/config.go                       | Before adding new config fields              |
| internal/verifier/provider_types.go               | Before adding new providers                  |
| internal/verifier/startup.go                      | Before modifying discovery/verification flow |
| internal/services/llmsverifier_score_adapter.go   | Before modifying scoring bridge              |
| docs/guides/llms-verifier.md                      | Before writing user-facing documentation     |
| docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md | Before starting any verifier work            |

---

## ## 5. Configuration Documentation

### ### 5.1 Full YAML Schema for `configs/verifier.yaml`

```
```yaml
# configs/verifier.yaml – LLMsVerifier Configuration Schema
```

[illegible]

```
# are marked failed
# Default: true

code_visibility_prompt: "Do you see my code?" # string – Prompt for c
# Default: "Do you see my code?"

verification_timeout: 60s # duration – Max time per verif
# Default: 60s

retry_count: 3 # int – Number of retries on t
# Default: 3

retry_delay: 5s # duration – Delay between retriee
# Default: 5s

max_concurrent: 5 # int – Max parallel verificatio
# Default: 5

tests: # []string – Enabled verification
- existence # Model exists at provider API
- responsiveness # Model responds to prompt
- latency # Response latency measured
- streaming # Streaming support verified
- function_calling # Tool/function calling verifie
- coding_capability # Coding task success verified
- error_detection # Error handling verified
- code_visibility # Code visibility confirmed

stale_threshold: 24h # duration – Max age before re-ver
# Default: 24h

# -----
# SECTION 4: Scoring Configuration
# -----
scoring:
weights: # map[string]float64 – MUST sum t
response_speed: 0.25 # Weight for latency/responsive
model_efficiency: 0.20 # Weight for throughput/tokens-p
cost_effectiveness: 0.25 # Weight for price performance
capability: 0.20 # Weight for feature richness
recency: 0.10 # Weight for model release date
# (newer models score higher)

models_dev_enabled: true # bool – Enable models.dev pric
# Default: true

models_dev_endpoint: "https://api.models.dev" # string – Pricing data
# Default: https://api.models.de

cache_ttl: 24h # duration – Score cache TTL
# Default: 24h

min_score: 5.0 # float64 – Minimum score for auto
# Models below this are not a
```

```

# but still displayed with war
# Default: 5.0
# Range: 0.0 - 10.0

# -----
# SECTION 5: Health Monitoring
# -----
health:
  check_interval: 30s          # duration — Health check interval
                                # Default: 30s

  timeout: 10s                 # duration — Health check timeout
                                # Default: 10s

  failure_threshold: 5         # int — Consecutive failures bef
                                # Default: 5

  recovery_threshold: 3        # int — Consecutive successes be
                                # Default: 3

  circuit_breaker:
    enabled: true              # bool — Enable circuit breaker
                                # Default: true

    half_open_timeout: 60s     # duration — Time in half-open before
                                # Default: 60s

# -----
# SECTION 6: API Server Configuration
# -----
api:
  enabled: true                # bool — Enable verifier REST API
                                # Default: true

  port: "8081"                 # string — API server port
                                # Default: 8081
                                # Env: HELIX_VERIFIER_API_PORT

  base_path: "/api/v1/verifier" # string — API base path
                                # Default: /api/v1/verifier

  host: "0.0.0.0"              # string — Bind host
                                # Default: 0.0.0.0

  jwt_secret: "${VERIFIER_JWT_SECRET}" # string — JWT signing secret
                                # Default: ""
                                # Env: VERIFIER_JWT_SECRET

  tls:                          # TLS configuration
    enabled: false              # bool — Enable TLS
    cert_file: ""               # string — Certificate path
    key_file: ""                # string — Key path

  rate_limit:                  # Rate limiting

```



```

    enabled: true                # bool
    requests_per_minute: 100    # int
    burst_size: 20              # int

cors:                          # CORS configuration
    enabled: true               # bool
    allowed_origins: ["*"]      # []string

# -----
# SECTION 7: Event System
# -----
events:
    websocket:
        enabled: true           # bool    – Enable WebSocket event str
                                # Default: true

        path: "/ws/verifier/events" # string – WebSocket endpoint path
                                # Default: /ws/verifier/events

        ping_interval: 30s      # duration – Keep-alive ping interval
                                # Default: 30s

    slack:
        enabled: false          # bool
        webhook_url: "${SLACK_WEBHOOK_URL}" # string
                                # Env: SLACK_WEBHOOK_URL

    email:
        enabled: false          # bool
        smtp_host: "smtp.gmail.com" # string
        smtp_port: 587          # int
        smtp_user: ""           # string
        smtp_password: ""       # string
        from_address: ""        # string

    telegram:
        enabled: false          # bool
        bot_token: "${TELEGRAM_BOT_TOKEN}" # string
                                # Env: TELEGRAM_BOT_TOKEN

        chat_id: "${TELEGRAM_CHAT_ID}"     # string
                                # Env: TELEGRAM_CHAT_ID

# -----
# SECTION 8: Monitoring
# -----
monitoring:
    prometheus:
        enabled: true           # bool
        path: "/metrics/verifier" # string
        port: "9091"           # string

    grafana:
        enabled: true           # bool

```

```

    dashboard_path: "./dashboards/verifier" # string

jaeger:                                     # Distributed tracing
  enabled: false                             # bool
  endpoint: ""                             # string

# -----
# SECTION 9: Brotli / HTTP3
# -----
brotli:
  enabled: true                             # bool    - Enable Brotli compression
                                          # Default: true

  compression_level: 6                     # int      - Brotli compression level
                                          # Default: 6

http3:
  enabled: false                             # bool    - Enable HTTP/3 (QUIC)
                                          # Default: false
                                          # Requires quic-go dependency

# -----
# SECTION 10: Challenges
# -----
challenges:
  enabled: true                             # bool    - Enable challenge system
                                          # Default: true

  provider_discovery: true                 # bool    - Run provider discovery ch
  model_verification: true                 # bool    - Run model verification c
  config_generation: true                  # bool    - Run config generation ch

# -----
# SECTION 11: Scheduling
# -----
scheduling:
  re_verification:
    enabled: true                           # bool    - Auto re-verify models peri
                                          # Default: true

    interval: 24h                          # duration - Re-verification interval
                                          # Default: 24h

    jitter: 1h                             # duration - Random jitter to avoid t
                                          # Default: 1h

  score_recalculation:
    enabled: true                           # bool    - Auto recalculate scores
                                          # Default: true

    interval: 12h                          # duration - Score recalculation inte
                                          # Default: 12h

  stale_model_cleanup:

```

```

    enabled: true                                # bool    – Remove models not seen in
                                                # Default: true

    max_age: 30d                                # duration – Max age before removal
                                                # Default: 30d

# -----
# SECTION 12: Provider Configuration
# -----
# Each provider has: enabled, api_key, base_url, models, timeout, retry
# Models list is OPTIONAL – if omitted, all models from provider are discovered:
openai:
    enabled: true
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    models: [] # empty = auto-discover all
    timeout: 30s
    retry: 3
    # Env: OPENAI_API_KEY, HELIX_OPENAI_API_KEY

anthropic:
    enabled: true
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
    models: []
    timeout: 30s
    retry: 3
    oauth_fallback: true # Fall back to OAuth if API key fails
    # Env: ANTHROPIC_API_KEY, CLAUDE_API_KEY, HELIX_ANTHROPIC_API_KEY

gemini:
    enabled: true
    api_key: "${GEMINI_API_KEY}"
    base_url: "https://generativelanguage.googleapis.com/v1beta"
    models: []
    timeout: 30s
    retry: 3
    # Env: GEMINI_API_KEY, GOOGLE_API_KEY, ApiKey_Gemini, HELIX_GEMINI_API_KEY

deepseek:
    enabled: true
    api_key: "${DEEPSEEK_API_KEY}"
    base_url: "https://api.deepseek.com/v1"
    models: []
    timeout: 30s
    retry: 3
    # Env: DEEPSEEK_API_KEY, HELIX_DEEPSEEK_API_KEY

groq:
    enabled: true
    api_key: "${GROQ_API_KEY}"
    base_url: "https://api.groq.com/openai/v1"
    models: []

```

```
    timeout: 30s
    retry: 3
    # Env: GROQ_API_KEY, HELIX_GROQ_API_KEY

together:
    enabled: true
    api_key: "${TOGETHER_API_KEY}"
    base_url: "https://api.together.xyz/v1"
    models: []
    timeout: 30s
    retry: 3
    # Env: TOGETHER_API_KEY, HELIX_TOGETHER_API_KEY

mistral:
    enabled: true
    api_key: "${MISTRAL_API_KEY}"
    base_url: "https://api.mistral.ai/v1"
    models: []
    timeout: 30s
    retry: 3
    # Env: MISTRAL_API_KEY, HELIX_MISTRAL_API_KEY

xai:
    enabled: true
    api_key: "${XAI_API_KEY}"
    base_url: "https://api.x.ai/v1"
    models: []
    timeout: 30s
    retry: 3
    # Env: XAI_API_KEY, HELIX_XAI_API_KEY

cerebras:
    enabled: false # Disabled by default (requires enterprise key)
    api_key: "${CEREBRAS_API_KEY}"
    base_url: "https://api.cerebras.ai/v1"
    models: []
    timeout: 30s
    retry: 3
    # Env: CEREBRAS_API_KEY, HELIX_CEREBRAS_API_KEY

cloudflare:
    enabled: true
    api_key: "${CLOUDFLARE_API_KEY}"
    account_id: "${CLOUDFLARE_ACCOUNT_ID}"
    base_url: "https://api.cloudflare.com/client/v4"
    models: []
    timeout: 30s
    retry: 3
    # Env: CLOUDFLARE_API_KEY, CLOUDFLARE_ACCOUNT_ID, HELIX_CLOUDFLARE_A

siliconflow:
    enabled: true
    api_key: "${SILICONFLOW_API_KEY}"
    base_url: "https://api.siliconflow.cn/v1"
```

```

models: []
timeout: 30s
retry: 3
# Env: SILICONFLOW_API_KEY, HELIX_SILICONFLOW_API_KEY

replicate:
  enabled: true
  api_token: "${REPLICATE_API_TOKEN}"
  base_url: "https://api.replicate.com/v1"
  models: []
  timeout: 60s
  retry: 3
  # Env: REPLICATE_API_TOKEN, HELIX_REPLICATE_API_TOKEN

openrouter:
  enabled: true
  api_key: "${OPENROUTER_API_KEY}"
  base_url: "https://openrouter.ai/api/v1"
  models: []
  timeout: 30s
  retry: 3
  free_models_only: false # Set true to use only free tier
  # Env: OPENROUTER_API_KEY, HELIX_OPENROUTER_API_KEY

qwen:
  enabled: true
  api_key: "${QWEN_API_KEY}"
  base_url: "https://dashscope.aliyuncs.com/api/v1"
  models: []
  timeout: 30s
  retry: 3
  oauth_primary: true # Prefer OAuth over API key
  # Env: QWEN_API_KEY, HELIX_QWEN_API_KEY

cohere:
  enabled: true
  api_key: "${COHERE_API_KEY}"
  base_url: "https://api.cohere.com/v1"
  models: []
  timeout: 30s
  retry: 3
  # Env: COHERE_API_KEY, HELIX_COHERE_API_KEY

ollama:
  enabled: true
  host: "http://localhost:11434" # Ollama server address
  models: [] # Auto-discover from /api/tags
  timeout: 120s
  # No API key needed for local

llamacpp:
  enabled: true
  host: "http://localhost:8080" # llama.cpp server address
  models: []

```

```

    timeout: 120s
    # No API key needed for local

vllm:
    enabled: false                    # Disabled by default
    host: "http://localhost:8000"
    models: []
    timeout: 120s

localai:
    enabled: false
    host: "http://localhost:8080"
    models: []
    timeout: 120s

```

## 5.2 Environment Variable Mapping Table

| Config Path                        | YAML Key       | Env Var (Primary)       |
|------------------------------------|----------------|-------------------------|
| verifier.enabled                   | enabled        | —                       |
| verifier.database.path             | path           | —                       |
| verifier.database.encryption_key   | encryption_key | VERIFIER_ENCRYPTION_KEY |
| verifier.api.port                  | port           | VERIFIER_API_PORT       |
| verifier.api.jwt_secret            | jwt_secret     | VERIFIER_JWT_SECRET     |
| verifier.events.slack.webhook_url  | webhook_url    | SLACK_WEBHOOK_URL       |
| verifier.events.telegram.bot_token | bot_token      | TELEGRAM_BOT_TOKEN      |
| verifier.events.telegram.chat_id   | chat_id        | TELEGRAM_CHAT_ID        |
| providers.openai.api_key           | api_key        | OPENAI_API_KEY          |
| providers.anthropic.api_key        | api_key        | ANTHROPIC_API_KEY       |
| providers.gemini.api_key           | api_key        | GEMINI_API_KEY          |
| providers.deepseek.api_key         | api_key        | DEEPSEEK_API_KEY        |
| providers.groq.api_key             | api_key        | GROQ_API_KEY            |
| providers.together.api_key         | api_key        | TOGETHER_API_KEY        |
| providers.mistral.api_key          | api_key        | MISTRAL_API_KEY         |
| providers.xai.api_key              | api_key        | XAI_API_KEY             |
| providers.cerebras.api_key         | api_key        | CEREBRAS_API_KEY        |
| providers.cloudflare.api_key       | api_key        | CLOUDFLARE_API_KEY      |
| providers.cloudflare.account_id    | account_id     | CLOUDFLARE_ACCOUNT_ID   |
| providers.siliconflow.api_key      | api_key        | SILICONFLOW_API_KEY     |
| providers.replicate.api_token      | api_token      | REPLICATE_API_TOKEN     |
| providers.openrouter.api_key       | api_key        | OPENROUTER_API_KEY      |

| Config Path              | YAML Key | Env Var (Primary) |
|--------------------------|----------|-------------------|
| providers.qwen.api_key   | api_key  | QWEN_API_KEY      |
| providers.cohere.api_key | api_key  | COHERE_API_KEY    |
| providers.ollama.host    | host     | OLLAMA_HOST       |
| providers.llamacpp.host  | host     | LLAMA_CPP_HOST    |

### 5.3 HelixCode Config Integration (internal/config/config.go additions)

The existing Config struct in internal/config/config.go must be extended:

```
type Config struct {
    Version      string    `mapstructure:"version"`
    UpdatedBy    string    `mapstructure:"updated_by"`
    Application  ApplicationConfig `mapstructure:"application"`
    Server       ServerConfig `mapstructure:"server"`
    Database     database.Config `mapstructure:"database"`
    Redis        RedisConfig `mapstructure:"redis"`
    Auth         AuthConfig `mapstructure:"auth"`
    Workers      WorkersConfig `mapstructure:"workers"`
    Tasks        TasksConfig `mapstructure:"tasks"`
    LLM          LLMConfig `mapstructure:"llm"`
    Providers    ProvidersConfig `mapstructure:"providers"`
    Logging      LoggingConfig `mapstructure:"logging"`
    Cognee       *CogneeConfig `mapstructure:"cognee"`
    Verifier     *VerifierConfig `mapstructure:"verifier" //
    NEW
}
```

```
// VerifierConfig is embedded from the verifier package config
// but mapped with HELIX_ prefix for env var binding.
type VerifierConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    Endpoint   string `mapstructure:"endpoint" // Verifier
    API endpoint
    APIKey     string `mapstructure:"api_key" // For
    authenticating TO the verifier
    Timeout    string `mapstructure:"timeout"`
    DatabasePath string `mapstructure:"database_path"`
}
```

**Env var binding additions** in config.go Load():

```
// Add to existing explicit binds:
viper.BindEnv("verifier.enabled", "HELIX_VERIFIER_ENABLED")
```

```

viper.BindEnv("verifier.endpoint", "HELIX_VERIFIER_ENDPOINT")
viper.BindEnv("verifier.api_key", "HELIX_VERIFIER_API_KEY")
viper.BindEnv("verifier.timeout", "HELIX_VERIFIER_TIMEOUT")
viper.BindEnv("verifier.database_path",
              "HELIX_VERIFIER_DATABASE_PATH")

// Provider API keys (already partially present, ensure complete
// set):
viper.BindEnv("providers.openai.api_key", "HELIX_OPENAI_API_KEY")
viper.BindEnv("providers.anthropic.api_key",
              "HELIX_ANTHROPIC_API_KEY")
viper.BindEnv("providers.gemini.api_key", "HELIX_GEMINI_API_KEY")
viper.BindEnv("providers.deepseek.api_key",
              "HELIX_DEEPSEEK_API_KEY")
viper.BindEnv("providers.groq.api_key", "HELIX_GROQ_API_KEY")
viper.BindEnv("providers.together.api_key",
              "HELIX_TOGETHER_API_KEY")
viper.BindEnv("providers.mistral.api_key",
              "HELIX_MISTRAL_API_KEY")
viper.BindEnv("providers.xai.api_key", "HELIX_XAI_API_KEY")
viper.BindEnv("providers.cerebras.api_key",
              "HELIX_CEREBRAS_API_KEY")
viper.BindEnv("providers.cloudflare.api_key",
              "HELIX_CLOUDFLARE_API_KEY")
viper.BindEnv("providers.cloudflare.account_id",
              "HELIX_CLOUDFLARE_ACCOUNT_ID")
viper.BindEnv("providers.siliconflow.api_key",
              "HELIX_SILICONFLOW_API_KEY")
viper.BindEnv("providers.replicate.api_token",
              "HELIX_REPLICATE_API_TOKEN")
viper.BindEnv("providers.openrouter.api_key",
              "HELIX_OPENROUTER_API_KEY")
viper.BindEnv("providers.qwen.api_key", "HELIX_QWEN_API_KEY")
viper.BindEnv("providers.cohere.api_key", "HELIX_COHERE_API_KEY")
viper.BindEnv("providers.ollama.host", "HELIX_OLLAMA_HOST")
viper.BindEnv("providers.llamacpp.host", "HELIX_LLAMA_CPP_HOST")

```

## 5.4 Example Configuration Files

### Basic Config (config.basic.yaml)

```

version: "1.0.0"

application:
  name: "helixcode"
  environment: "production"

server:

```



```
port: 8080
host: "0.0.0.0"

database:
  host: "localhost"
  port: 5432
  name: "helixcode"
  user: "helix"
  password: "${HELIX_DATABASE_PASSWORD}"
```

```
llm:
  default_provider: "openai"
  default_model: "gpt-4o"
  max_tokens: 4096
  temperature: 0.7
```

```
# Verifier — minimal configuration
```

```
verifier:
  enabled: true
  endpoint: "http://localhost:8081"
  timeout: "30s"
```

```
providers:
  openai:
    enabled: true
    api_key: "${HELIX_OPENAI_API_KEY}"
  anthropic:
    enabled: true
    api_key: "${HELIX_ANTHROPIC_API_KEY}"
  ollama:
    enabled: true
    host: "http://localhost:11434"
```

### **Development Config (config.development.yaml)**

```
version: "1.0.0-dev"
```

```
application:
  name: "helixcode"
  environment: "development"
```

```
server:
  port: 8080
  host: "127.0.0.1"
```

```
database:
  host: "localhost"
  port: 5432
```

```
name: "helixcode_dev"
user: "helix"
password: "dev_password"

llm:
  default_provider: "ollama"
  default_model: "llama3.2"
  max_tokens: 2048
  temperature: 0.8

# Verifier – development settings
verifier:
  enabled: true
  endpoint: "http://localhost:8081"
  timeout: "60s"
  database_path: "./data/llm-verifier-dev.db"

# Full verifier config for development
verifier_full:
  database:
    path: "./data/llm-verifier-dev.db"
    encryption_enabled: false
    wal_mode: true

  verification:
    mandatory_code_check: false # Faster in dev
    verification_timeout: 30s
    retry_count: 1
    tests:
      - existence
      - responsiveness
      - latency

  scoring:
    weights:
      response_speed: 0.40
      model_efficiency: 0.20
      cost_effectiveness: 0.20
      capability: 0.10
      recency: 0.10
    cache_ttl: 1h
    min_score: 3.0 # Lower threshold in dev

  health:
    check_interval: 10s
    failure_threshold: 10 # More lenient in dev

api:
```

```
enabled: true
port: "8081"
rate_limit:
  enabled: false # No rate limiting in dev

events:
  websocket:
    enabled: true
  slack:
    enabled: false
  email:
    enabled: false
  telegram:
    enabled: false

challenges:
  enabled: true

scheduling:
  re_verification:
    enabled: true
    interval: 1h # Re-verify frequently in dev
  score_recalculation:
    enabled: true
    interval: 30m

providers:
  openai:
    enabled: true
    api_key: "${HELIX_OPENAI_API_KEY}"
    timeout: 30s
    retry: 1
  anthropic:
    enabled: true
    api_key: "${HELIX_ANTHROPIC_API_KEY}"
    timeout: 30s
  deepseek:
    enabled: true
    api_key: "${HELIX_DEEPSEEK_API_KEY}"
  groq:
    enabled: true
    api_key: "${HELIX_GROQ_API_KEY}"
  ollama:
    enabled: true
    host: "http://localhost:11434"
  llamacpp:
    enabled: true
    host: "http://localhost:8080"
```

```
openrouter:
  enabled: true
  api_key: "${HELIX_OPENROUTER_API_KEY}"
  free_models_only: true # Use free tier in dev
```

### **Production Config** (config.production.yaml)

```
version: "1.0.0"

application:
  name: "helixcode"
  environment: "production"

server:
  port: 8080
  host: "0.0.0.0"

database:
  host: "${HELIX_DATABASE_HOST}"
  port: 5432
  name: "helixcode"
  user: "helix"
  password: "${HELIX_DATABASE_PASSWORD}"
  ssl_mode: "require"
  max_connections: 50

redis:
  host: "${HELIX_REDIS_HOST}"
  port: "${HELIX_REDIS_PORT}"
  password: "${HELIX_REDIS_PASSWORD}"

llm:
  default_provider: "openai"
  default_model: "gpt-4o"
  max_tokens: 4096
  temperature: 0.7

# Verifier – production settings
verifier:
  enabled: true
  endpoint: "http://verifier.internal:8081" # Internal service
  discovery
  timeout: "30s"
  database_path: "/data/llm-verifier.db"

verifier_full:
  database:
    path: "/data/llm-verifier.db"
```

```
encryption_enabled: true
encryption_key: "${VERIFIER_ENCRYPTION_KEY}"
wal_mode: true
max_connections: 25

verification:
  mandatory_code_check: true
  verification_timeout: 60s
  retry_count: 3
  retry_delay: 5s
  max_concurrent: 10
  tests:
    - existence
    - responsiveness
    - latency
    - streaming
    - function_calling
    - coding_capability
    - error_detection
    - code_visibility
  stale_threshold: 24h

scoring:
  weights:
    response_speed: 0.25
    model_efficiency: 0.20
    cost_effectiveness: 0.25
    capability: 0.20
    recency: 0.10
  cache_ttl: 24h
  min_score: 6.0

health:
  check_interval: 30s
  timeout: 10s
  failure_threshold: 5
  recovery_threshold: 3
  circuit_breaker:
    enabled: true
    half_open_timeout: 60s

api:
  enabled: true
  port: "8081"
  base_path: "/api/v1/verifier"
  jwt_secret: "${VERIFIER_JWT_SECRET}"
  tls:
    enabled: true
```

```
    cert_file: "/etc/helixcode/certs/verifier.crt"
    key_file: "/etc/helixcode/certs/verifier.key"
rate_limit:
    enabled: true
    requests_per_minute: 1000
    burst_size: 100

events:
    websocket:
        enabled: true
        ping_interval: 30s
    slack:
        enabled: true
        webhook_url: "${SLACK_WEBHOOK_URL}"
    email:
        enabled: true
        smtp_host: "${SMTP_HOST}"
        smtp_port: 587
        smtp_user: "${SMTP_USER}"
        smtp_password: "${SMTP_PASSWORD}"
        from_address: "verifier@helixcode.dev"
    telegram:
        enabled: false

monitoring:
    prometheus:
        enabled: true
        path: "/metrics/verifier"
        port: "9091"
    grafana:
        enabled: true
        dashboard_path: "/etc/helixcode/dashboards/verifier"

brotli:
    enabled: true
    compression_level: 6

challenges:
    enabled: true

scheduling:
    re_verification:
        enabled: true
        interval: 24h
        jitter: 1h
    score_recalculation:
        enabled: true
        interval: 12h
```

```
    stale_model_cleanup:
      enabled: true
      max_age: 30d

providers:
  openai:
    enabled: true
    api_key: "${HELIX_OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    timeout: 30s
    retry: 3
  anthropic:
    enabled: true
    api_key: "${HELIX_ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
    timeout: 30s
    retry: 3
    oauth_fallback: true
  gemini:
    enabled: true
    api_key: "${HELIX_GEMINI_API_KEY}"
    base_url: "https://generativelanguage.googleapis.com/v1beta"
    timeout: 30s
    retry: 3
  deepseek:
    enabled: true
    api_key: "${HELIX_DEEPSEEK_API_KEY}"
    base_url: "https://api.deepseek.com/v1"
    timeout: 30s
    retry: 3
  groq:
    enabled: true
    api_key: "${HELIX_GROQ_API_KEY}"
    base_url: "https://api.groq.com/openai/v1"
    timeout: 30s
    retry: 3
  mistral:
    enabled: true
    api_key: "${HELIX_MISTRAL_API_KEY}"
    base_url: "https://api.mistral.ai/v1"
    timeout: 30s
    retry: 3
  xai:
    enabled: true
    api_key: "${HELIX_XAI_API_KEY}"
    base_url: "https://api.x.ai/v1"
    timeout: 30s
    retry: 3
```

```
openrouter:
  enabled: true
  api_key: "${HELIX_OPENROUTER_API_KEY}"
  base_url: "https://openrouter.ai/api/v1"
  timeout: 30s
  retry: 3
ollama:
  enabled: false # Disabled in production (use cloud
    providers)
  host: "http://localhost:11434"
```

### Testing Config (config.testing.yaml)

```
version: "1.0.0-test"
```

```
application:
  name: "helixcode"
  environment: "testing"
```

```
server:
  port: 18080
  host: "127.0.0.1"
```

```
database:
  host: "localhost"
  port: 5432
  name: "helixcode_test"
  user: "helix"
  password: "test_password"
```

```
llm:
  default_provider: "openrouter"
  default_model: "openai/gpt-3.5-turbo"
  max_tokens: 1024
  temperature: 0.7
```

```
# Verifier — testing settings (uses mock verifier or test fixture
  DB)
```

```
verifier:
  enabled: true
  endpoint: "http://localhost:18081" # Test verifier instance
  timeout: "10s"
  database_path: ":memory:" # In-memory SQLite for tests
```

```
verifier_full:
  database:
    path: ":memory:"
    encryption_enabled: false
```



```
verification:
  mandatory_code_check: false
  verification_timeout: 10s
  retry_count: 1
  tests:
    - existence
    - responsiveness
```

```
scoring:
  weights:
    response_speed: 0.25
    model_efficiency: 0.20
    cost_effectiveness: 0.25
    capability: 0.20
    recency: 0.10
  cache_ttl: 5m
  min_score: 1.0 # Very low threshold for testing
```

```
health:
  check_interval: 5s
  failure_threshold: 100 # Never trip in tests
```

```
api:
  enabled: true
  port: "18081"
  rate_limit:
    enabled: false
```

```
events:
  websocket:
    enabled: false # No WebSocket in tests
```

```
challenges:
  enabled: false # Challenges run separately
```

```
scheduling:
  re_verification:
    enabled: false # Manual re-verify only in tests
```

```
providers:
  openrouter:
    enabled: true
    api_key: "${HELIX_OPENROUTER_API_KEY}"
    free_models_only: true # Use free tier for tests
    timeout: 10s
    retry: 1
  ollama:
```

```
enabled: true
host: "http://localhost:11434"
```

---

## 6. User Guide

### 6.1 How to Enable/Disable LLMsVerifier

**Enable** (default):

```
# In config.yaml
verifier:
  enabled: true
```

Or via environment variable:

```
export HELIX_VERIFIER_ENABLED=true
```

**Disable:**

```
# In config.yaml
verifier:
  enabled: false
```

When disabled: - ModelManager falls back to legacy behavior (provider self-discovery) - - -  
list-models shows only what providers report directly - No scoring, no verification status, no  
circuit breaker - Hardcoded models (if any) become visible again — this is a degradation

**Verify Status:**

```
helixcode verifier status
# Output: ENABLED | Endpoint: http://localhost:8081 |
          Database: ./data/llm-verifier.db
```

### 6.2 How to Configure API Keys for All Providers

**Method 1: Environment Variables** (Recommended for security)

```
# Required for cloud providers
export HELIX_OPENAI_API_KEY="sk-..."
export HELIX_ANTHROPIC_API_KEY="sk-ant-..."
export HELIX_GEMINI_API_KEY="AIza..."

# Optional — only if you use these providers
export HELIX_DEEPSEEK_API_KEY="..."
export HELIX_GROQ_API_KEY="..."
export HELIX_MISTRAL_API_KEY="..."
export HELIX_XAI_API_KEY="..."
export HELIX_OPENROUTER_API_KEY="..."
```

```
# Local providers need no keys
# export HELIX_OLLAMA_HOST="http://localhost:11434" # Only if
# non-default
```

**Method 2: Config File** (Convenient for development only — NEVER commit to git)

```
providers:
  openai:
    enabled: true
    api_key: "sk-..." # DANGER: Never commit this file
```

**Method 3: OAuth (Anthropic, Qwen)**

```
# OAuth tokens are read from secure storage (keychain / OS
# credential store)
# No env var needed if OAuth is configured
# Set oauth_primary: true in config to prefer OAuth over API key
```

**Verify Keys Are Detected:**

```
helixcode verifier providers list --format json
# Shows "key_status": "available" or "missing" for each provider
```

**6.3 How to Read Model Listings and Interpret Status Indicators**

**CLI Listing:**

```
helixcode --list-models
# or
helixcode models list --format table
```

**Sample Output:**

| # | Model                    | Provider  | Score | Status     | Latency |
|---|--------------------------|-----------|-------|------------|---------|
| 1 | GPT-4o SC:9.2            | OpenAI    | 9.2   | ✓ Ready    | 245ms   |
| 2 | Claude 3.5 Sonnet SC:9.0 | Anthropic | 9.0   | ✓ Ready    | 312ms   |
| 3 | Gemini 2.5 Pro SC:8.8    | Gemini    | 8.8   | ✓ Ready    | 189ms   |
| 4 | DeepSeek Coder SC:8.5    | DeepSeek  | 8.5   | ✓ Ready    | 420ms   |
| 5 | Llama 3.2 3B SC:7.1      | Ollama    | 7.1   | ✓ Ready    | 156ms   |
| 6 | GPT-3.5 Turbo SC:7.8     | OpenAI    | 7.8   | ○ Stale    | unknown |
| 7 | Mistral Large SC:8.2     | Mistral   | 8.2   | ✗ Failed   | --      |
| 8 | Grok-2 SC:8.0            | xAI       | 8.0   | 🕒 Cooldown | 5m rer  |

Legend: ✓ Verified   ○ Stale (>24h)   ✗ Failed   🕒 Rate Limited / Cooldown

**Interpreting Status:**

| Status  | Meaning | Action                  |
|---------|---------|-------------------------|
| ✓ Ready |         | Available for selection |

| Status       | Meaning                                | Action                                              |
|--------------|----------------------------------------|-----------------------------------------------------|
|              | Verified within 24h, score > min_score |                                                     |
| ○ Stale      | Last verified >24h ago                 | Will be re-verified; still usable but deprioritized |
| × Failed     | Verification test failed               | Not available for selection                         |
| 🕒 Cooldown   | Rate limit or temporary ban            | Wait for cooldown or select alternative             |
| 🔄 Verifying  | Verification in progress               | Check again in a few seconds                        |
| ❓ Deprecated | Provider marked model deprecated       | Manual override required to use                     |

### JSON Output for Scripting:

```
helixcode models list --format json | jq '.models[] |
  select(.status == "ready") | .id'
```

## 6.4 How to Handle Cooldown / Rate-Limited Models

**Automatic Handling:** - `SelectOptimalModel()` automatically excludes rate-limited models - The circuit breaker opens after `failure_threshold` consecutive failures - Fallback to next-highest-scored available model

### Manual Override:

```
# Force use of a specific model even if rate-limited
helixcode --prompt "hello" --model gpt-4o --force
# Warning: may fail with rate limit error
```

### Check Cooldown Status:

```
helixcode verifier limits list --provider openai
# Shows: remaining_requests, reset_time, limit_type
```

### Wait and Retry:

```
# Poll until model is ready
while helixcode models get gpt-4o --format json | jq -e
  '.status != "ready"' > /dev/null; do
  echo "Waiting for gpt-4o..."
  sleep 30
done
```

## 6.5 Troubleshooting Guide

**Symptom:** `--list-models` shows empty list

### Diagnosis:

# Step 1: Check if verifier is running

```
helixcode verifier health
```

# Step 2: Check verifier logs

```
tail -f logs/verifier.log
```

# Step 3: Check if providers are configured

```
helixcode verifier config show | grep -A5 providers
```

# Step 4: Check if API keys are detected

```
helixcode verifier providers list --verbose
```

**Common Causes:** 1. Verifier server not started → `helixcode verifier start` 2. `verifier.enabled: false` in config → Set to `true` 3. No API keys configured → Set env vars 4. Verifier database corrupted → `rm data/llm-verifier.db` and restart 5. Network issues → Check `ping api.openai.com`

**Symptom:** `--list-models` shows hardcoded 3 models (BLUFF-002)

**This is a Constitutional Violation.** The code is using the old `handleListModel()` instead of the verifier.

**Fix:** Confirm `internal/verifier/discovery.go` is integrated and `cmd/cli/main.go:handleListModel()` calls `discoveryService.GetVerifiedModels()`.

**Symptom:** Model score is always 8.5

**Diagnosis:** The verifier is using the `stub_verification.go` instead of `real_coding_capability_verification.go`.

**Fix:** Check LLMsVerifier submodule is at correct commit. Run `git submodule update --init --recursive`.

**Symptom:** Provider shows “available” but requests fail

**Diagnosis:**

# Check health status

```
helixcode verifier providers get openai --verbose
```

# Check circuit breaker state

```
helixcode verifier health --provider openai
```

# Try direct API call

```
curl -H "Authorization: Bearer $HELIX_OPENAI_API_KEY" https://api.openai.com/v1/models
```

**Common Causes:** 1. API key expired or revoked → Regenerate key 2. Account rate limit reached → Wait or upgrade plan 3. Provider API changed → Update verifier submodule 4. Circuit breaker stuck open → Wait for `half_open_timeout` or restart

**Symptom: Scores seem wrong / weights don't make sense**

**Diagnosis:**

```
# Check current weights
helixcode verifier config show --path scoring.weights

# Recalculate scores manually
helixcode verifier scoring recalculate --model gpt-4o --verbose
```

**Fix:** Edit configs/verifier.yaml scoring weights. Ensure they sum to 1.0.

**Symptom: OAuth provider (Claude) fails after working**

**Diagnosis:**

```
# Check OAuth token expiry
helixcode verifier oauth status --provider anthropic

# Check token refresh log
grep "oauth_refresh" logs/verifier.log
```

**Fix:** Re-authenticate via helixcode verifier oauth login --provider anthropic.

**Symptom: Verifier TUI shows different models than CLI**

**Diagnosis:** TUI and CLI may use different cache refresh intervals.

**Fix:** Press r in TUI to force refresh. Check events.websocket.enabled is true.

## 7. Integration Guide for Developers

### 7.1 How the Integration Works Architecturally

#### Layer Model

| User Interface Layer (CLI, TUI, Web, API)       |                                                   |
|-------------------------------------------------|---------------------------------------------------|
| cmd/cli/main.go                                 | → calls internal/verifier/service.go              |
| internal/server/                                | → calls internal/verifier/service.go              |
| applications/                                   | → calls pkg/sdk/go/verifier/client.go             |
| Application Layer                               |                                                   |
| internal/services/llmsverifier_score_adapter.go | → Bridges ProviderDiscovery with verifier scoring |
| internal/llm/model_manager.go                   | → Uses score adapter for SelectOptimalModel()     |

|                                                        |                         |
|--------------------------------------------------------|-------------------------|
| <b>Verifier Layer (HelixCode Wrapper)</b>              |                         |
| internal/verifier/service.go                           | → VerificationService   |
| internal/verifier/discovery.go                         | → ModelDiscoveryService |
| internal/verifier/startup.go                           | → StartupVerifier       |
| internal/verifier/scoring.go                           | → ScoringService        |
| internal/verifier/health.go                            | → HealthService         |
| internal/verifier/events.go                            | → EventPublisher        |
| <b>Verifier Layer (Submodule – LLMsVerifier)</b>       |                         |
| LLMsVerifier/llm-verifier/verification/verification.go |                         |
| LLMsVerifier/llm-verifier/providers/*.go               |                         |
| LLMsVerifier/llm-verifier/scoring/scoring_engine.go    |                         |
| LLMsVerifier/llm-verifier/api/server.go                |                         |
| <b>External APIs</b>                                   |                         |
| OpenAI, Anthropic, Gemini, DeepSeek, Groq, ...         |                         |

### Data Flow: Model Discovery

1. StartupVerifier.VerifyAllProviders() called at server startup
2. Phase 1: discoverProviders() → scans env vars, OAuth, local endpoints
3. Phase 2: verifyProviders() → calls each provider's API, runs verification tests
4. Phase 3: detectSubscriptions() → determines tier (Premium/High-quality/Fast)
5. Phase 4: scoreProviders() → runs 7-component scoring engine
6. Phase 5: rankProviders() → sorts by score, selects debate team
7. Results stored in SQLite database (data/llm-verifier.db)
8. VerificationService exposes results to HelixCode application layer
9. LLMsVerifierScoreAdapter bridges to ModelManager.SelectOptimalModel()

### Data Flow: Real-Time Update

1. Provider status changes (rate limit, failure, recovery)
2. HealthService detects change via periodic checks
3. EventPublisher publishes VerificationEvent to WebSocket
4. HelixCode server subscribes to WebSocket
5. TUI/CLI receives event and updates display
6. If WebSocket unavailable, polling fallback every 30s

## 7.2 How to Extend LLMsVerifier with New Providers

### Step-by-Step

#### 1. Add Provider to LLMsVerifier Submodule

In LLMsVerifier/llm-verifier/providers/:

```
// newprovider.go
package providers

type NewProviderAdapter struct {
    BaseAdapter
}

func NewNewProviderAdapter(endpoint, apiKey string)
    *NewProviderAdapter {
    return &NewProviderAdapter{
        BaseAdapter: BaseAdapter{
            client:    &http.Client{Timeout: 30 * time.Second},
            endpoint: endpoint,
            apiKey:    apiKey,
            headers: map[string]string{
                "Authorization": "Bearer " + apiKey,
                "Content-Type": "application/json",
            },
        },
    }
}

func (a *NewProviderAdapter) SendRequest(ctx context.Context,
    req *LLMRequest) (*http.Response, error) {
    body, _ := json.Marshal(req)
    httpReq, _ := http.NewRequestWithContext(ctx, "POST",
        a.endpoint+"/chat/completions", bytes.NewReader(body))
    for k, v := range a.headers {
        httpReq.Header.Set(k, v)
    }
    return a.client.Do(httpReq)
}

func (a *NewProviderAdapter) ParseResponse(resp *http.Response)
    (*LLMResponse, error) {
    // Parse provider-specific response format
}
```

## 2. Add Fallback Models

In `LLMsVerifier/llm-verifier/providers/fallback_models.go`:

```
var NewProviderFallbackModels = []string{
    "model-1",
    "model-2",
}
```

## 3. Register in HelixCode

In `internal/verifier/provider_types.go`:



```
"newprovider": {
  Type:      "newprovider",
  DisplayName: "New Provider",
  AuthType:   AuthTypeAPIKey,
  Tier:       2,
  Priority:    5,
  EnvVars:    []string{"NEWPROVIDER_API_KEY",
    "HELIX_NEWPROVIDER_API_KEY"},
  BaseURL:    "https://api.newprovider.com/v1",
  Models:     []string{"model-1", "model-2"},
}
```

#### 4. Add Config Section

In configs/verifier.yaml:

```
providers:
  newprovider:
    enabled: true
    api_key: "${HELIX_NEWPROVIDER_API_KEY}"
    base_url: "https://api.newprovider.com/v1"
    models: []
    timeout: 30s
    retry: 3
```

#### 5. Add Tests

```
// internal/verifier/adapters/newprovider_test.go
func TestNewProviderAdapter_SendRequest(t *testing.T) {
    // Unit test with httptest server
}

// tests/integration/newprovider_verification_test.go
func TestNewProvider_Verification_Integration(t *testing.T) {
    if os.Getenv("HELIX_SKIP_LIVE_PROVIDER_TESTS") != "" {
        t.Skip("SKIP-OK: Live tests disabled")
    }
    // Real API call
}
```

#### 6. Add Challenge

```
# challenges/scripts/newprovider_challenge.sh
#!/bin/bash
set -e

echo "=== NewProvider Challenge ==="

# Verify provider is discoverable
```

```
./helixcode verifier providers list --format json | jq -e '[] |
  select(.type == "newprovider")'

# Verify models are found
./helixcode verifier models list --provider newprovider --format
  json | jq -e '.models | length > 0'

# Verify a model responds
./helixcode --prompt "Hello" --model model-1 --provider
  newprovider

echo "=== NewProvider Challenge PASSED ==="
```

## 7. Update Documentation

- Add to docs/guides/llms-verifier.md provider list
- Add to .env.example
- Add to this document's "Minimum Provider Set" table

## 7.3 How to Modify Scoring Weights

### 1. Edit Config:

```
scoring:
  weights:
    response_speed: 0.35      # Increased from 0.25
    model_efficiency: 0.15    # Decreased from 0.20
    cost_effectiveness: 0.20  # Decreased from 0.25
    capability: 0.20          # Unchanged
    recency: 0.10             # Unchanged
```

### 2. Validate (must sum to 1.0):

```
helixcode verifier config validate
# Error if weights != 1.0
```

### 3. Recalculate Scores:

```
helixcode verifier scoring recalculate --all
# Re-runs scoring for all models with new weights
```

### 4. Verify:

```
helixcode models list --sort score --format json | jq
  '.models[0:3] | map(.id, .score)'
# Confirm top models changed appropriately
```

**Weight Guidelines:** - **Latency-sensitive workloads** (chat, autocomplete): Increase response\_speed, decrease cost\_effectiveness - **Cost-sensitive workloads** (batch processing): Increase cost\_effectiveness, decrease response\_speed - **Capability-critical workloads** (code generation, complex reasoning): Increase capability - **Always keep recency at 5-10%** to avoid permanently selecting obsolete models

## 7.4 How to Add New Model Capabilities

### 1. Add Capability Flag to Verifier Database

In `LLMsVerifier/llm-verifier/database/database.go`, add column:

```
ALTER TABLE models ADD COLUMN supports_new_capability BOOLEAN
    DEFAULT 0;
ALTER TABLE verification_results ADD COLUMN
    new_capability_tested BOOLEAN DEFAULT 0;
```

### 2. Add Verification Test

In `LLMsVerifier/llm-verifier/verification/coding_capability_verification.go` or new file:

```
func (v *Verifier) TestNewCapability(ctx context.Context,
    modelID string, client ProviderClientInterface) (bool,
    error) {
    prompt := "Test prompt for new capability..."
    response, err := client.SendPrompt(ctx, modelID, prompt)
    if err != nil {
        return false, err
    }
    return evaluateNewCapability(response), nil
}
```

### 3. Add to HelixCode Capability Mapping

In `internal/verifier/provider_types.go`:

```
type UnifiedModel struct {
    // ... existing fields ...
    SupportsNewCapability bool `json:"supports_new_capability"`
}
```

### 4. Use in Model Selection

In `internal/llm/model_manager.go`:

```
func (mm *ModelManager) SelectOptimalModel(criteria
    ModelSelectionCriteria) (*ModelInfo, error) {
    // ... existing filtering ...
    if criteria.RequiresNewCapability {
        candidates = filterByNewCapability(candidates)
    }
}
```

### 5. Update CONST-040 Checklist

Add new checkbox:

- [ ] New capability verified for at least 2 providers

## 7.5 API Reference for Internal Verifier Client

Go SDK (pkg/sdk/go/verifier/client.go)

```
package verifier

// Client communicates with the LLMsVerifier REST API
type Client struct {
    baseURL    string
    apiKey     string
    httpClient *http.Client
}

// ClientConfig configures the client
type ClientConfig struct {
    BaseURL    string // Default: http://localhost:8081
    APIKey     string // JWT token for verifier API auth
    Timeout    time.Duration // Default: 30s
    HTTPClient *http.Client // Optional custom HTTP client
}

// New creates a new verifier client
func New(cfg ClientConfig) *Client

// VerifyModel runs verification for a single model
// POST /api/v1/verifier/models/{id}/verify
func (c *Client) VerifyModel(ctx context.Context, req
    VerificationRequest) (*VerificationResult, error)

// BatchVerify runs verification for multiple models
// POST /api/v1/verifier/batch/verify
func (c *Client) BatchVerify(ctx context.Context, req
    BatchVerifyRequest) (*BatchVerifyResult, error)

// GetModels returns all models from the verifier database
// GET /api/v1/verifier/models
func (c *Client) GetModels(ctx context.Context, filter
    ModelFilter) ([]*UnifiedModel, error)

// GetModel returns a single model by ID
// GET /api/v1/verifier/models/{id}
func (c *Client) GetModel(ctx context.Context, modelID string)
    (*UnifiedModel, error)

// GetProviderScores returns provider-level scores
// GET /api/v1/verifier/providers/scores
```

```

func (c *Client) GetProviderScores(ctx context.Context)
    (map[string]float64, error)

// GetModelScores returns model-level scores
// GET /api/v1/verifier/models/scores
func (c *Client) GetModelScores(ctx context.Context)
    (map[string]float64, error)

// GetProviders returns all providers with metadata
// GET /api/v1/verifier/providers
func (c *Client) GetProviders(ctx context.Context)
    ([]*UnifiedProvider, error)

// GetHealth returns health status for all providers
// GET /api/v1/verifier/health
func (c *Client) GetHealth(ctx context.Context) (*HealthStatus,
    error)

// GetLimits returns rate limits for all models
// GET /api/v1/verifier/limits
func (c *Client) GetLimits(ctx context.Context) ([]*RateLimit,
    error)

// GetEvents returns recent verification events
// GET /api/v1/verifier/events
func (c *Client) GetEvents(ctx context.Context, limit int)
    ([]*VerificationEvent, error)

// SubscribeEvents opens a WebSocket connection for real-time
    events
// WS /ws/verifier/events
func (c *Client) SubscribeEvents(ctx context.Context)
    (*EventStream, error)

// ExportConfig exports verifier configuration in specified
    format
// GET /api/v1/verifier/config/export?format={yaml|json|toml}
func (c *Client) ExportConfig(ctx context.Context, format
    string) ([]byte, error)

```

### VerificationRequest / VerificationResult

```

type VerificationRequest struct {
    ModelID      string          `json:"model_id"`
    Provider     string          `json:"provider"`
    Tests        []string        `json:"tests,omitempty"` //
        Subset of tests to run
    Timeout      time.Duration   `json:"timeout,omitempty"`

```

```

}

type VerificationResult struct {
    ModelID          string          `json:"model_id"`
    Provider          string          `json:"provider"`
    Status            string          `json:"status" //
        pending, running, completed, failed
    ModelExists       *bool
        `json:"model_exists,omitempty"`
    Responsive         *bool
        `json:"responsive,omitempty"`
    Overloaded         *bool
        `json:"overloaded,omitempty"`
    LatencyMs          int64           `json:"latency_ms"`
    SupportsToolUse    bool
        `json:"supports_tool_use"`
    SupportsCodeGeneration bool
        `json:"supports_code_generation"`
    SupportsStreaming  bool
        `json:"supports_streaming"`
    SupportsReasoning  bool
        `json:"supports_reasoning"`
    OverallScore        float64         `json:"overall_score"`
    CodeCapabilityScore float64
        `json:"code_capability_score"`
    ResponsivenessScore float64
        `json:"responsiveness_score"`
    ReliabilityScore    float64
        `json:"reliability_score"`
    FeatureRichnessScore float64
        `json:"feature_richness_score"`
    ValuePropositionScore float64
        `json:"value_proposition_score"`
    Timestamp           time.Time       `json:"timestamp"`
    Error               string
        `json:"error,omitempty"`
}

```

### WebSocket Event Stream

```

type EventStream struct {
    conn *websocket.Conn
}

// Read reads the next event from the stream
func (s *EventStream) Read() (*VerificationEvent, error)

// Close closes the WebSocket connection

```

```
func (s *EventStream) Close() error

type VerificationEvent struct {
    Type          VerificationEventType `json:"type"`
    Provider      string                 `json:"provider,omitempty"`
    ModelID       string                 `json:"model_id,omitempty"`
    Score         float64                `json:"score,omitempty"`
    Status        string                 `json:"status,omitempty"`
    Timestamp     time.Time              `json:"timestamp"`
    Data          map[string]interface{} `json:"data,omitempty"`
}
```

---

## 8. Submodule Constitution Template

This section provides a **template** for applying all LLMsVerifier constitutional rules to every submodule. Each submodule **MUST** have its own `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. The rules below **MUST** be inserted into those files.

### 8.1 Submodule `CONSTITUTION.md` Insertion Template

**Insert the following block into every submodule's `CONSTITUTION.md` after the highest-numbered existing rule** (e.g., if the submodule has rules up to `CONST-020`, insert after `CONST-020`; if it has up to `CONST-035`, insert after `CONST-035`):

---

```
### CONST-SUB-001: LLMsVerifier Single Source of Truth Mandate
    (Submodule Edition)
```

```
**Rule**: This submodule SHALL treat LLMsVerifier as the
    authoritative source for all model, provider, and
    capability metadata that it consumes or exposes. No
    submodule may maintain its own hardcoded model or
    provider registry independent of the verifier.
```

```
**Scope**: This rule applies regardless of whether the submodule
    directly imports `digital.vasic.llmsverifier`. If the
    submodule uses models (e.g., for testing, benchmarking,
    or CLI interaction), those models MUST be sourced from
    the verifier's published data, not from local constants.
```

```
**Verification**: The submodule's `make test-complete` MUST
    include a check that any model referenced in tests exists
    in the verifier database or is explicitly annotated with
    `// verifier:verified`.
```

---

### ### CONST-SUB-002: Anti-Bluff Testing Guarantee (Submodule Edition)

**\*\*Rule\*\***: The "Tests Pass But Features Don't Work" guarantee applies WITH FULL FORCE to this submodule. No test may pass unless the feature it exercises is demonstrably usable in the built artifact.

**\*\*Specific Prohibitions\*\***:

- Mocking the verifier with hardcoded scores in integration or E2E tests
- Asserting on hardcoded expected output when the real output should come from a provider API
- Skipping live provider tests by default (they may be skipped with ``SKIP-OK`` markers when env vars are absent)

---

### ### CONST-SUB-003: Submodule Constitution Propagation Requirement

**\*\*Rule\*\***: If this submodule has its own submodules (nested), the three rules above (CONST-SUB-001, CONST-SUB-002, CONST-SUB-003) MUST be propagated to those nested submodules as well.

**\*\*Enforcement\*\***: The presence of these rules in a submodule's ``CONSTITUTION.md`` is verified by ``challenges/scripts/submodule_constitution_check.sh`` which scans all ``CONSTITUTION.md`` files in the repository tree.

## 8.2 Submodule CLAUDE.md Insertion Template

Insert the following section into every submodule's CLAUDE.md:

---

### ## LLMsVerifier Integration Guidelines for This Submodule

#### ### Context

This submodule is part of the Helix ecosystem. LLMsVerifier is the single source of truth for all model and provider metadata. The main project constitution mandates:

- CONST-036: LLMsVerifier Single Source of Truth
- CONST-037: Model Provider Anti-Bluff Guarantee
- CONST-038: Real-Time Model Status Accuracy
- CONST-039: All Providers and Models Integration
- CONST-040: MCP/LSP/ACP/Embedding/RAG/Skills/Plugins Integration

#### ### What This Submodule Must Do



1. **\*\*If this submodule uses LLMs\*\***: All model references MUST be verifier-aware. Do not hardcode model names.
2. **\*\*If this submodule tests LLMs\*\***: Tests MUST use real provider APIs or the verifier database, not mocks (above unit test tier).
3. **\*\*If this submodule provides a model interface\*\***: It MUST integrate with ``internal/verifier/`` or ``pkg/sdk/go/verifier``.

#### ### What This Submodule Must NOT Do

1. Maintain its own model registry independent of the verifier
2. Return hardcoded model lists to users
3. Mock verifier responses in non-unit tests
4. Skip provider capability verification

#### ### Integration Points

- For Go submodules: Import ``github.com/HelixDevelopment/helix_code/pkg/sdk/go/verifier``
- For non-Go submodules: Use the REST API at ``http://localhost:8081/api/v1/verifier``
- For test submodules: Use the test fixture database at ``:memory:`` or a test-specific SQLite file

#### ### Verification Checklist for Submodule Developers

- [ ] No hardcoded model names in source code
- [ ] No hardcoded provider endpoints (use verifier config)
- [ ] Integration tests use real APIs or verifier DB
- [ ] All model references can be traced to verifier data

### 8.3 Submodule AGENTS.md Insertion Template

**Insert the following section into every submodule's AGENTS.md:**

---

#### ## LLMsVerifier-Aware Development Rules

##### ### BLUFF-SUB-001: Submodule Has Independent Model Registry (CRITICAL)

- \*\*Pattern\*\***: Submodule contains a ``models.go``, ``registry.go``, or similar file with hardcoded ``[]string`` of model names.
- \*\*Fix\*\***: Replace with verifier client call or import from ``pkg/sdk/go/verifier``.

##### ### BLUFF-SUB-002: Submodule Tests Use Hardcoded Expected Output (HIGH)

- \*\*Pattern\*\***: Test asserts ``expected := "GPT-4 is a model by OpenAI"`` when testing model metadata.
- \*\*Fix\*\***: Assert on verifier database content instead.

```
### BLUFF-SUB-003: Submodule Bypasses Verifier for Provider
    Selection (HIGH)
**Pattern**: Submodule implements its own provider selection
    logic that does not consult verifier scores.
**Fix**: Use `LLMsVerifierScoreAdapter.GetProviderScore()` or
    equivalent.
```

### ### Technology Stack Note

If this submodule uses models, it depends on:

- `github.com/HelixDevelopment/helix\_code/pkg/sdk/go/verifier`  
(Go submodules)
- `http://localhost:8081/api/v1/verifier` (REST API for all  
languages)
- `digital.vasic.llmsverifier` (direct import, Go only)

### ### Required Files Checklist

If this submodule interacts with LLMs, these files MUST exist:

- [ ] `docs/LLMsVERIFIER\_INTEGRATION.md` (how this submodule uses  
the verifier)
- [ ] `\*\_test.go` with verifier-aware tests (for Go submodules)
- [ ] `.env.example` with required `HELIX\_\*` env vars (if  
applicable)

## ### 8.4 Submodule Verification Checklist

**For every submodule in the Helix ecosystem, run this checklist before declaring LLMsVerifier integration complete:**

### ## Submodule LLMsVerifier Compliance Verification

#### ### Documentation

- [ ] Submodule `CONSTITUTION.md` contains CONST-SUB-001, CONST-SUB-002, CONST-SUB-003
- [ ] Submodule `CLAUDE.md` contains "LLMsVerifier Integration Guidelines" section
- [ ] Submodule `AGENTS.md` contains "LLMsVerifier-Aware Development Rules" section
- [ ] Submodule has `docs/LLMsVERIFIER\_INTEGRATION.md` (even if brief)

#### ### Code

- [ ] `grep -r "llama-3-8b\|mistral-7b\|phi-3-mini" --include="\*.go" --include="\*.py" --include="\*.js"` returns zero results (or only in verifier-related files)
- [ ] No `[ ]string{` model arrays in non-test, non-verifier code
- [ ] No hardcoded provider endpoints (e.g., `"https://api.openai.com"` as a string literal)

- [ ] Provider selection uses verifier scores (or falls back to a documented default)
- ### Tests
- [ ] Integration tests do not mock verifier responses
  - [ ] E2E tests use the real verifier database or real provider APIs
  - [ ] `make test` includes a verifier-aware test (even if it skips with `SKIP-OK`)
- ### Configuration
- [ ] `.env.example` lists all provider API keys this submodule needs
  - [ ] Config loader reads `HELIX_VERIFIER_*` env vars if verifier integration exists

---

## Appendix A: Line Number Insertion Guide

### HelixCode CONSTITUTION.md

| Section             | Insert After      | Approximate Content to Search For                                |
|---------------------|-------------------|------------------------------------------------------------------|
| CONST-036           | CONST-035 closing | ### CONST-035: End-User Usability Mandate closing paragraph      |
| CONST-037-040       | CONST-036         | Immediately after CONST-036 block                                |
| CONST-035 Amendment | Within CONST-035  | After “every PASS must guarantee quality, completion, usability” |
| CONST-017 Amendment | Within CONST-017  | After “Zero-Bluff Testing” main body                             |
| Appendix D          | End of document   | Before any - - - or end marker                                   |

### HelixCode CLAUDE.md

| Section                   | Insert After                | Search For                                        |
|---------------------------|-----------------------------|---------------------------------------------------|
| LLMsVerifier Architecture | Core Systems / Architecture | ## Architecture or ## Core Systems                |
| Direct Client Pattern     | Integration Patterns        | After Pattern 3 description                       |
| Add Provider Guide        | How to Add                  | After ### How to Add a New Provider or create new |
| API Key Provisioning      | Authentication section      | After JWT/auth section                            |
| Debugging                 | Troubleshooting             | After existing troubleshooting or create new      |

| Section           | Insert After   | Search For                             |
|-------------------|----------------|----------------------------------------|
| Real-Time Updates | Event System   | After internal/event/<br>documentation |
| UX Guidelines     | UI/CLI section | After ## CLI Usage or<br>equivalent    |

**HelixCode AGENTS.md**

| Section             | Insert After    | Search For                                          |
|---------------------|-----------------|-----------------------------------------------------|
| BLUFF-004           | BLUFF-003       | ### BLUFF-003:<br>Command Execution is<br>Simulated |
| BLUFF-005-008       | BLUFF-004       | After BLUFF-004 block                               |
| Tech Stack          | Existing stack  | #### Technology Stack<br>or ## Technology Stack     |
| Module Boundaries   | Architecture    | ## Module Boundaries<br>or ### Module<br>Boundaries |
| Challenge Checklist | End of document | Before any end marker                               |

**Appendix B: Environment Variable Quick Reference**

**All HELIX\_VERIFIER\_\* Variables**

```
# Core verifier
HELIX_VERIFIER_ENABLED=true|false
HELIX_VERIFIER_ENDPOINT=http://localhost:8081
HELIX_VERIFIER_API_KEY=<jwt-for-verifier-auth>
HELIX_VERIFIER_TIMEOUT=30s
HELIX_VERIFIER_DATABASE_PATH=./data/llm-verifier.db
HELIX_VERIFIER_ENCRYPTION_KEY=<sql-cipher-key>
HELIX_VERIFIER_JWT_SECRET=<verifier-jwt-signing-secret>
HELIX_VERIFIER_API_PORT=8081

# API keys for cloud providers
HELIX_OPENAI_API_KEY
HELIX_ANTHROPIC_API_KEY
HELIX_GEMINI_API_KEY
HELIX_DEEPSEEK_API_KEY
HELIX_GROQ_API_KEY
HELIX_TOGETHER_API_KEY
HELIX_MISTRAL_API_KEY
HELIX_XAI_API_KEY
HELIX_CEREBRAS_API_KEY
HELIX_CLOUDFLARE_API_KEY
```

```
HELIX_CLOUDFLARE_ACCOUNT_ID
HELIX_SILICONFLOW_API_KEY
HELIX_REPLICATE_API_TOKEN
HELIX_OPENROUTER_API_KEY
HELIX_QWEN_API_KEY
HELIX_COHERE_API_KEY

# Local providers
HELIX_OLLAMA_HOST=http://localhost:11434
HELIX_LLAMA_CPP_HOST=http://localhost:8080
HELIX_VLLM_HOST=http://localhost:8000
HELIX_LOCALAI_HOST=http://localhost:8080

# Test control
HELIX_SKIP_LIVE_PROVIDER_TESTS=1      # Set to skip live API tests
HELIX_VERIFIER_TEST_MODE=1            # Enable test mode (reduced
                                     verification)
```

## Appendix C: File Creation Checklist

For each file that MUST be created or modified:

| #  | File                              | Action                                                      | Status   |
|----|-----------------------------------|-------------------------------------------------------------|----------|
| 1  | CONSTITUTION.md                   | Amend with CONST-036-040, CONST-035/017 updates, Appendix D | Required |
| 2  | CLAUDE.md                         | Insert LLMsVerifier architecture section                    | Required |
| 3  | AGENTS.md                         | Insert BLUFF-004-008, module boundaries, checklist          | Required |
| 4  | internal/config/config.go         | Add VerifierConfig struct, env var bindings                 | Required |
| 5  | configs/verifier.yaml             | Create full schema                                          | Required |
| 6  | configs/verifier.basic.yaml       | Create example                                              | Required |
| 7  | configs/verifier.development.yaml | Create example                                              | Required |
| 8  | configs/verifier.production.yaml  | Create example                                              | Required |
| 9  | configs/verifier.testing.yaml     | Create example                                              | Required |
| 10 | .env.example                      | Add all HELIX_VERIFIER_* and provider keys                  | Required |

| #  | File                                                              | Action                                                                                                                          | Status   |
|----|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------|
| 11 | internal/verifier/service.go                                      | Create VerificationService wrapper                                                                                              | Required |
| 12 | internal/verifier/config.go                                       | Create Config structs                                                                                                           | Required |
| 13 | internal/verifier/discovery.go                                    | Create ModelDiscoveryService                                                                                                    | Required |
| 14 | internal/verifier/startup.go                                      | Create StartupVerifier                                                                                                          | Required |
| 15 | internal/verifier/scoring.go                                      | Create ScoringService adapter                                                                                                   | Required |
| 16 | internal/verifier/health.go                                       | Create HealthService                                                                                                            | Required |
| 17 | internal/verifier/events.go                                       | Create EventPublisher                                                                                                           | Required |
| 18 | internal/verifier/provider_types.go                               | Create SupportedProviders map                                                                                                   | Required |
| 19 | internal/services/llmsverifier_score_adapter.go                   | Create score bridge                                                                                                             | Required |
| 20 | pkg/sdk/go/verifier/client.go                                     | Create Go SDK client                                                                                                            | Required |
| 21 | docs/guides/llms-verifier.md                                      | Create user guide                                                                                                               | Required |
| 22 | docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md                 | Create developer plan                                                                                                           | Required |
| 23 | challenges/scripts/llmsverifier_hardcode_check.sh                 | Create hardcode scanner                                                                                                         | Required |
| 24 | challenges/scripts/llmsverifier_capabilities_challenge.sh         | Create capability test                                                                                                          | Required |
| 25 | challenges/scripts/llmsverifier_status_accuracy_challenge.sh      | Create status test                                                                                                              | Required |
| 26 | challenges/scripts/llmsverifier_startup_verification_challenge.sh | Create startup test                                                                                                             | Required |
| 27 | challenges/scripts/submodule_constitution_check.sh                | Create propagation check                                                                                                        | Required |
| 28 | Makefile                                                          | Add verifier test targets                                                                                                       | Required |
| 29 | LLMsVerifier/ (submodule)                                         | Add git submodule add <a href="https://github.com/vasic-digital/LLMsVerifier">https://github.com/vasic-digital/LLMsVerifier</a> | Required |
| 30 | cmd/cli/main.go                                                   | Replace hardcoded models with verifier fetch                                                                                    | Required |
| 31 | internal/llm/model_discovery.go                                   |                                                                                                                                 | Required |

| #  | File                          | Action                                | Status   |
|----|-------------------------------|---------------------------------------|----------|
|    |                               | Replace hardcoded fetchExternalModels |          |
| 32 | internal/llm/model_manager.go | Add verifier status to scoring        | Required |

## Appendix D: Cross-Reference to HelixAgent Implementation

For implementers, the following HelixAgent files are the canonical reference for each component:

| Component            | HelixAgent Reference File                                         | Lines | Purpose                |
|----------------------|-------------------------------------------------------------------|-------|------------------------|
| VerificationService  | internal/verifier/service.go                                      | 1097  | Core service           |
| Config               | internal/verifier/config.go                                       | 398   | Configuration          |
| Discovery            | internal/verifier/discovery.go                                    | 526   | Model discovery        |
| Startup              | internal/verifier/startup.go                                      | 1873  | Startup logic          |
| Provider Types       | internal/verifier/provider_types.go                               | 1043  | Unified provider types |
| Scoring              | internal/verifier/scoring.go                                      | 754   | Scoring logic          |
| Enhanced Scoring     | internal/verifier/enhanced_scoring.go                             | 730   | 7-component scoring    |
| Events               | internal/verifier/events.go                                       | 337   | Event processing       |
| Health               | internal/verifier/health.go                                       | 486   | Health checks          |
| Score Adapter        | internal/services/llmsverifier_score_adapter.go                   | 528   | Bridge to Provider     |
| Go SDK               | pkg/sdk/go/verifier/client.go                                     | 385   | Client interface       |
| Config YAML          | configs/verifier.yaml                                             | 257   | Full configuration     |
| Integration Guide    | docs/guides/llms-verifier.md                                      | 328   | User guide             |
| Integration Plan     | docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md                 | 2423  | 10-phase plan          |
| Power Features       | docs/verifier/LLMSVERIFIER_POWER_FEATURES.md                      | 661   | Advanced features      |
| Challenge: CLI       | challenges/scripts/llmsverifier_cliagents_challenge.sh            | 304   | CLI agent challenge    |
| Challenge: Startup   | challenges/scripts/llmsverifier_startup_verification_challenge.sh | 138   | Startup challenge      |
| Challenge: Submodule | challenges/scripts/llmsverifier_submodule_smoke_challenge.sh      | 126   | Submodule challenge    |
| Test Suite           | tests/helixllm/llmsverifier_test_suite.sh                         | 531   | Full test suite        |

# End of Document

**Legal Force Statement:** Every rule, amendment, and mandate in this document carries the full constitutional authority of the Helix project. Implementation is not optional. The anti-bluff guarantees (CONST-037, CONST-017 LLMsVerifier clause) are binding on all developers, all agents, and all automated systems. A test that passes while the feature it tests is non-functional is a constitutional violation, not a technical debt item.

**Submodule Propagation:** The template in Section 8 MUST be applied to every submodule without exception. The `submodule_constitution_check.sh` challenge script verifies compliance.

**Version:** 1.0.0-Draft

**Effective Upon:** Merge approval by project governance

**Review Cycle:** Every 30 days or upon verifier schema change